

使用 Google ADK 建構正式環境 AI 程式碼審查助理

來源：<https://codelabs.developers.google.com/adk-code-reviewer-assistant/instructions?hl=zh-tw>

步驟數：9

從 AI 程式設計工具的使用者，變身為創作者。在本實作程式碼研究室中，您將使用 Google Agent Development Kit (ADK)，建構適用於正式環境的 AI 程式碼審查助理。您將建構精密的多代理系統，提供比簡單的 LLM 意見回饋更深入的資訊。這個系統會使用確定性工具分析程式碼結構、執行實際測試，並驗證樣式是否符合規定。接著，您將建構反覆修正管道，自動修正已識別的問題、驗證變更，並重試，直到程式碼完美為止。最後，您將學會如何將這個完整的代理程式部署至 Google Cloud，並全面掌握正式環境的觀測資料。

目錄

1. [1. The Late Night Code Review](#)
2. [2. 首次部署代理程式](#)
3. [3. 準備正式版工作區](#)
4. [4. 建立第一個代理程式](#)
5. [5. 建構管道：多個代理協同運作](#)
6. [6. 新增修正管道：迴圈架構](#)
7. [7. 部署至正式環境](#)
8. [8. 正式版觀測](#)
9. [9. 結論：從原型設計到正式環境](#)

1. The Late Night Code Review

現在是凌晨 2 點

您已進行數小時的偵錯作業，函式看起來沒問題，但就是無法正常運作。您一定有過這種感覺：程式碼應該可以運作，但就是不行，而且您盯著程式碼太久，已經看不出原因了。

```
def dfs_search_v1(graph, start, target):  
    """Find if target is reachable from start."""  
    visited = set()  
    stack = start # Looks innocent enough...  
  
    while stack:  
        current = stack.pop()  
  
        if current == target:  
            return True  
  
        if current not in visited:  
            visited.add(current)  
  
            for neighbor in graph[current]:  
                if neighbor not in visited:  
                    stack.append(neighbor)  
  
    return False
```

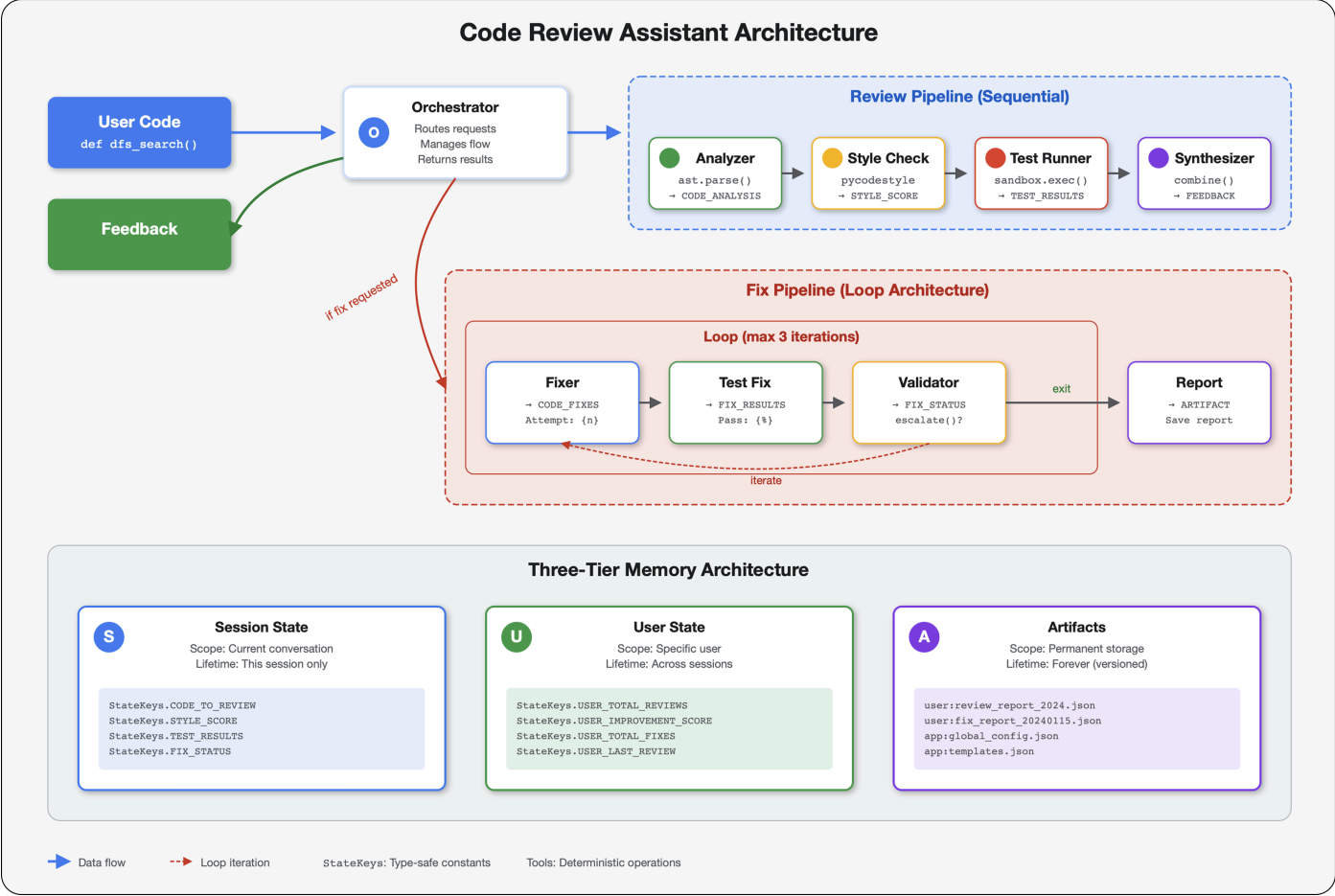
AI 開發人員旅程

如果您正在閱讀本文，可能已體驗過 AI 為程式設計帶來的轉變。[Gemini Code Assist](#)、[Claude Code](#) 和 [Cursor](#) 等工具改變了我們編寫程式碼的方式。這類工具非常適合產生樣板、建議實作方式，以及加快開發速度。

但您想深入瞭解，您想瞭解如何**建構**這些 AI 系統，而不只是使用。您想建立的內容應符合下列條件：

- 行為可預測且可追蹤
- 可安心部署到正式環境
- 提供穩定一致的結果，值得信賴
- 準確顯示 AI 的決策方式

從消費者到創作者



今天，您將從使用 AI 工具，進階到建構 AI 工具。您將建構多代理系統，該系統會：

1. 確定性地分析程式碼結構
2. 執行實際測試，驗證行為
3. 使用實際的 **Linter** 驗證樣式是否符合規定
4. 統整調查結果，提供實用回饋
5. 部署至 Google Cloud，並提供完整的可觀測性

步驟 2 · 約 0 分鐘

2. 首次部署代理程式

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

開發人員的問題

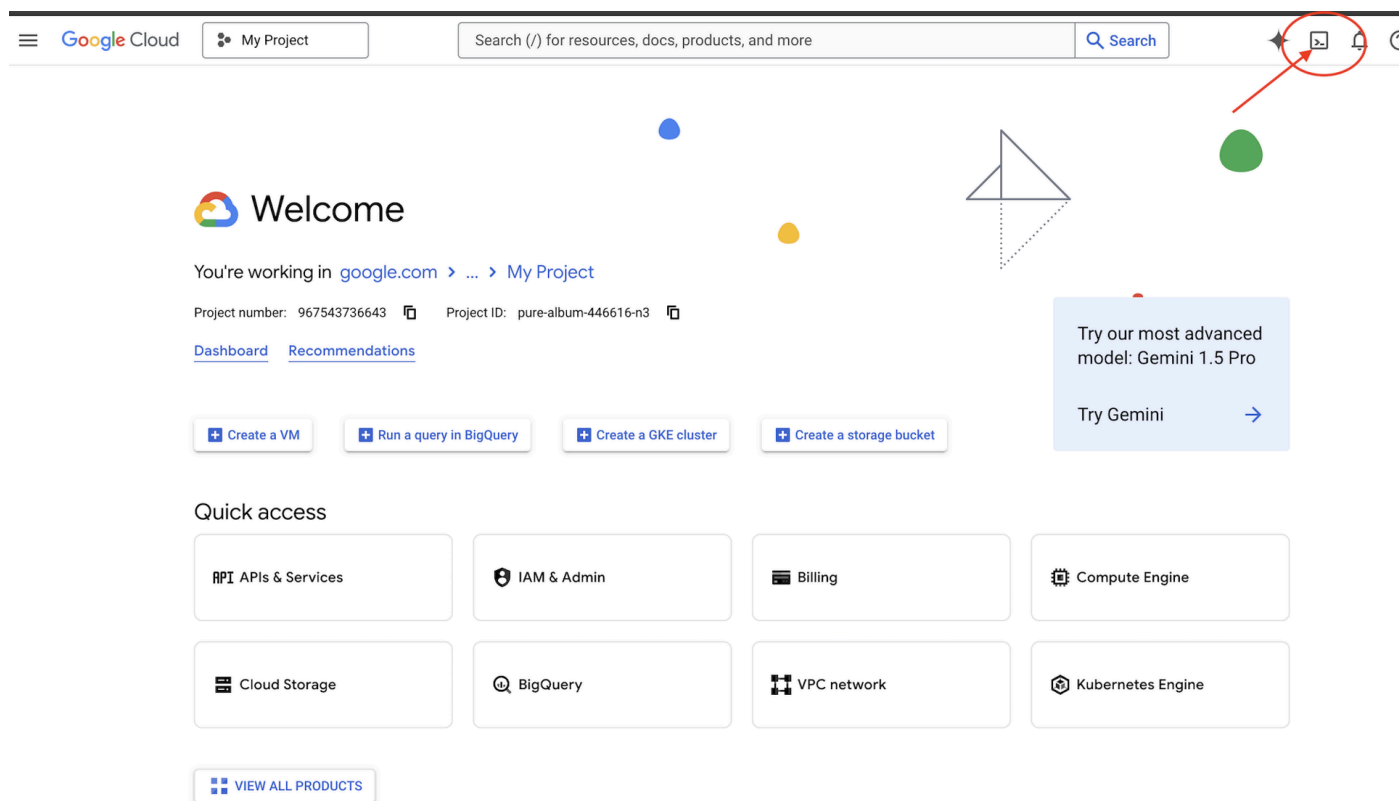
「我瞭解 LLM，也用過 API，但如何從 Python 指令碼變成可擴充的正式版 AI 代理程式？」

讓我們正確設定環境，然後建構簡單的代理程式，瞭解基本概念，再深入探討正式版模式。

請先完成基本設定

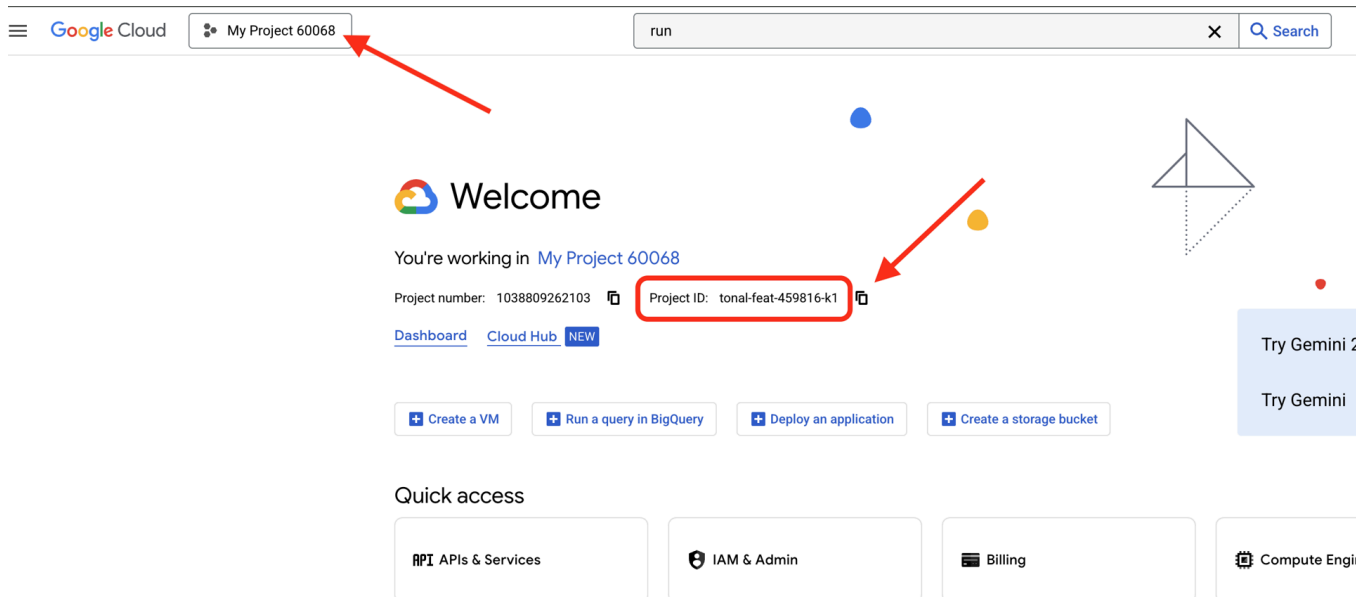
建立任何代理程式前，請先確認 Google Cloud 環境已就緒。

點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」(這是 Cloud Shell 窗格頂端的終端機形狀圖示)，



找出 Google Cloud 專案 ID：

- 開啟 Google Cloud 控制台：<https://console.cloud.google.com>
- 在頁面頂端的專案下拉式選單中，選取要用於本研討會的專案。
- 專案 ID 會顯示在資訊主頁的「專案資訊」資訊卡中



步驟 1：設定專案 ID

在 Cloud Shell 中，`gcloud` 指令列工具已設定完成。執行下列指令，設定有效專案。這會使用 `$GOOGLE_CLOUD_PROJECT` 環境變數，系統會在 Cloud Shell 工作階段中自動為您設定這個變數。

```
gcloud config set project $GOOGLE_CLOUD_PROJECT
```

注意：雖然上述指令是 Cloud Shell 中最直接的方法，但您也可以視需要手動設定專案 ID，方法是將 `$GOOGLE_CLOUD_PROJECT` 替換為特定 ID (例如 `gcloud config set project your-project-id-123`)。

步驟 2：驗證設定

接下來，請執行下列指令，確認專案設定正確無誤，且您已通過驗證。

```
# Confirm project is set
echo "Current project: $(gcloud config get-value project)"

# Check authentication status
gcloud auth list
```

您應該會看到列印的專案 ID，以及旁邊有 `(ACTIVE)` 的使用者帳戶。

如果帳戶未列為有效帳戶，或您收到驗證錯誤訊息，請執行下列指令登入：

```
gcloud auth application-default login
```

步驟 3：啟用必要 API

基本代理程式至少需要下列 API：

```
gcloud services enable \
  aiplatform.googleapis.com \
  compute.googleapis.com
```

這可能需要幾分鐘的時間。您會看到：

```
Operation "operations/..." finished successfully.
```

步驟 4：安裝 ADK

```
# Install the ADK CLI
pip install google-adk --upgrade

# Verify installation
adk --version
```

您應該會看到 1.15.0 以上的版本號碼。

現在建立基本代理程式

環境準備就緒後，我們來建立簡單的代理程式。

步驟 5：使用 ADK Create

```
adk create my_first_agent
```

按照互動式提示操作：

```
Choose a model for the root agent:
1. gemini-2.5-flash
2. Other models (fill later)
Choose model (1, 2): 1

1. Google AI
2. Vertex AI
Choose a backend (1, 2): 2

Enter Google Cloud project ID [auto-detected-from-gcloud]:
Enter Google Cloud region [us-central1]:
```

注意：專案 ID 應會從 `gcloud config` 自動填入。按下 Enter 鍵即可接受。同樣地，除非有特定偏好設定，否則請按下 Enter 鍵接受預設區域。

步驟 6：檢查建立的內容

```
cd my_first_agent
ls -la
```

您會看到三個檔案：

```
.env          # Configuration (auto-populated with your project)
__init__.py   # Package marker
agent.py      # Your agent definition
```

步驟 7：快速檢查設定

```
# Verify the .env was created correctly
cat .env

# Should show something like:
# GOOGLE_CLOUD_PROJECT=your-project-id
# GOOGLE_CLOUD_LOCATION=us-central1
# GOOGLE_GENAI_USE_VERTEXAI=1
```

如果專案 ID 遺漏或不正確，請編輯 `.env` 檔案：

```
nano .env # or use your preferred editor
```

步驟 8：查看代理程式碼

```
cat agent.py
```

```
from google.adk.agents.llm_agent import Agent

root_agent = Agent(
    model='gemini-2.5-flash',
    name='root_agent',
    description='A helpful assistant for user questions.',
    instruction='Answer user questions to the best of your knowledge',
)
```

簡單、乾淨、極簡。這是代理程式的「Hello World」。

測試基本代理程式

步驟 9：執行代理

```
cd ..
adk run my_first_agent
```

您應該會看到類似下方的內容：

```
Log setup complete: /tmp/agents_log/agent.20250930_162430.log
To access latest log: tail -F /tmp/agents_log/agent.latest.log
[user]:
```

步驟 10：嘗試查詢

在執行 `adk run` 的終端機中，您會看到提示。輸入查詢：

```
[user]: Hey, what can you do?
[root_agent]: I can help answer questions, generate text, explain concepts, assist with
problem-solving, and provide information on a wide range of topics. What would you like to
know or discuss?

[user]: What's the current weather in Mountain View?
[root_agent]: I'm sorry, but I cannot provide real-time weather information as I don't have
access to current data or weather services. To get the current weather in Mountain View, I
recommend checking a weather website or app like weather.com or using a search engine.
```

請注意這項限制，因為這項工具無法存取目前的資料。讓我們進一步探討：

```
[user]: Review this Python code: def add(a, b): return a + b
[root_agent]: This is a simple Python function that adds two numbers. Here's my review:

**Strengths:**
- Clean and concise implementation
- Follows Python conventions for simple operations
- Clear function name that describes its purpose

**Suggestions for improvement:**
- Add a docstring to document the function
- Consider adding type hints for better code clarity
- You might want to handle edge cases

Here's an enhanced version:
def add(a: float, b: float) -> float:
    """Add two numbers and return their sum."""
    return a + b
```

注意：抽象語法樹 (AST) 是原始碼語法結構的階層式樹狀結構表示法。程式設計語言剖析器讀取程式碼時，會將線性文字轉換為樹狀結構，擷取程式碼元素之間的文法關係。

代理可以討論程式碼，但可以：

- 實際剖析 AST 以瞭解結構？
- 執行測試來驗證是否正常運作？
- 檢查樣式是否符合規定？
- 還記得先前的評論嗎？

否。這時就需要架構。

  使用

Ctrl+C

探索完畢後。

步驟 3 · 約 0 分鐘

3. 準備正式版工作區

解決方法：可投入實作環境的架構

簡單代理程式展示了起點，但生產系統需要穩固的結構。現在，我們將設定一個完整的專案，體現生產原則。

設定基礎

您已為基本代理程式設定 Google Cloud 雲端專案。現在，我們要準備完整的正式環境工作區，其中包含實際系統所需的所有工具、模式和基礎架構。

步驟 1：取得結構化專案

首先，請使用 `Ctrl+C` 結束所有正在執行的 `adk run`，然後進行清理：

```
# Clean up the basic agent
cd ~ # Make sure you're not inside my_first_agent
rm -rf my_first_agent

# Get the production scaffold
git clone https://github.com/ayoisio/adk-code-review-assistant.git
cd adk-code-review-assistant
git checkout codelab
```

注意：codelab 分支版本包含完整的專案結構，以及策略性預留位置 (例如 `# MODULE_3_STEP_1_REPLACE_ME`)，您將在學習過程中新增程式碼。

步驟 2：建立並啟用虛擬環境

```
# Create the virtual environment
python -m venv .venv

# Activate it
# On macOS/Linux:
source .venv/bin/activate
# On Windows:
# .venv\Scripts\activate
```

驗證：提示現在應該會以 `(.venv)` 開頭。

步驟 3：安裝依附元件

```
pip install -r code_review_assistant/requirements.txt

# Install the package in editable mode (enables imports)
pip install -e .
```

這會安裝下列項目：

- `google-adk` - ADK 架構
- `pycodestyle` - 用於 PEP 8 檢查
- `vertexai` - 適用於雲端部署作業
- 其他生產依附元件

`-e` 標記可讓您從任何位置匯入 `code_review_assistant` 模組。

步驟 4：設定環境

```
# Copy the example environment file
cp .env.example .env

# Edit .env and replace the placeholders:
# - GOOGLE_CLOUD_PROJECT=your-project-id → your actual project ID
# - Keep other defaults as-is
```

驗證：檢查設定：

```
cat .env
```

應該顯示：

```
GOOGLE_CLOUD_PROJECT=your-actual-project-id
GOOGLE_CLOUD_LOCATION=us-central1
GOOGLE_GENAI_USE_VERTEXAI=TRUE
```

步驟 5：確認驗證

由於您先前已執行 `gcloud auth`，請驗證下列事項：

```
# Check current authentication
gcloud auth list

# Should show your account with (ACTIVE)
# If not, run:
gcloud auth application-default login
```

步驟 6：啟用其他正式版 API

我們已啟用基本 API。現在新增正式版：


```
gcloud services enable \  
  sqladmin.googleapis.com \  
  run.googleapis.com \  
  cloudbuild.googleapis.com \  
  artifactregistry.googleapis.com \  
  storage.googleapis.com \  
  cloudtrace.googleapis.com
```

這項做法可提供以下優勢：

- **SQL 管理員**：使用 Cloud Run 時適用於 Cloud SQL
- **Cloud Run**：用於無伺服器部署作業
- **Cloud Build**：用於自動化部署
- **Artifact Registry**：適用於容器映像檔
- **Cloud Storage**：用於構件和暫存
- **Cloud Trace**：用於可觀測性

步驟 7：建立 Artifact Registry 存放區

我們的部署作業會建構需要主目錄的容器映像檔：

```
gcloud artifacts repositories create code-review-assistant-repo \  
  --repository-format=docker \  
  --location=us-central1 \  
  --description="Docker repository for Code Review Assistant"
```

如下所示：

```
Created repository [code-review-assistant-repo].
```

如果該目錄已存在 (可能來自先前的嘗試)，請忽略您看到的錯誤訊息。

步驟 8：授予 IAM 權限

實務人員注意事項：在第 7 模組中部署代理程式時使用的 `deploy.sh` 指令碼，會使用 Cloud Build 自動執行部署作業。我們必須授予服務帳戶必要權限，這是重要的正式環境安全步驟。

```
# Get your project number
PROJECT_NUMBER=$(gcloud projects describe $GOOGLE_CLOUD_PROJECT \
  --format="value(projectNumber)")

# Define the service account
SERVICE_ACCOUNT="${PROJECT_NUMBER}@cloudbuild.gserviceaccount.com"

# Grant necessary roles
gcloud projects add-iam-policy-binding $GOOGLE_CLOUD_PROJECT \
  --member="serviceAccount:${SERVICE_ACCOUNT}" \
  --role="roles/run.admin"

gcloud projects add-iam-policy-binding $GOOGLE_CLOUD_PROJECT \
  --member="serviceAccount:${SERVICE_ACCOUNT}" \
  --role="roles/iam.serviceAccountUser"

gcloud projects add-iam-policy-binding $GOOGLE_CLOUD_PROJECT \
  --member="serviceAccount:${SERVICE_ACCOUNT}" \
  --role="roles/cloudsql.admin"

gcloud projects add-iam-policy-binding $GOOGLE_CLOUD_PROJECT \
  --member="serviceAccount:${SERVICE_ACCOUNT}" \
  --role="roles/storage.admin"
```

每個指令都會輸出：

```
Updated IAM policy for project [your-project-id].
```

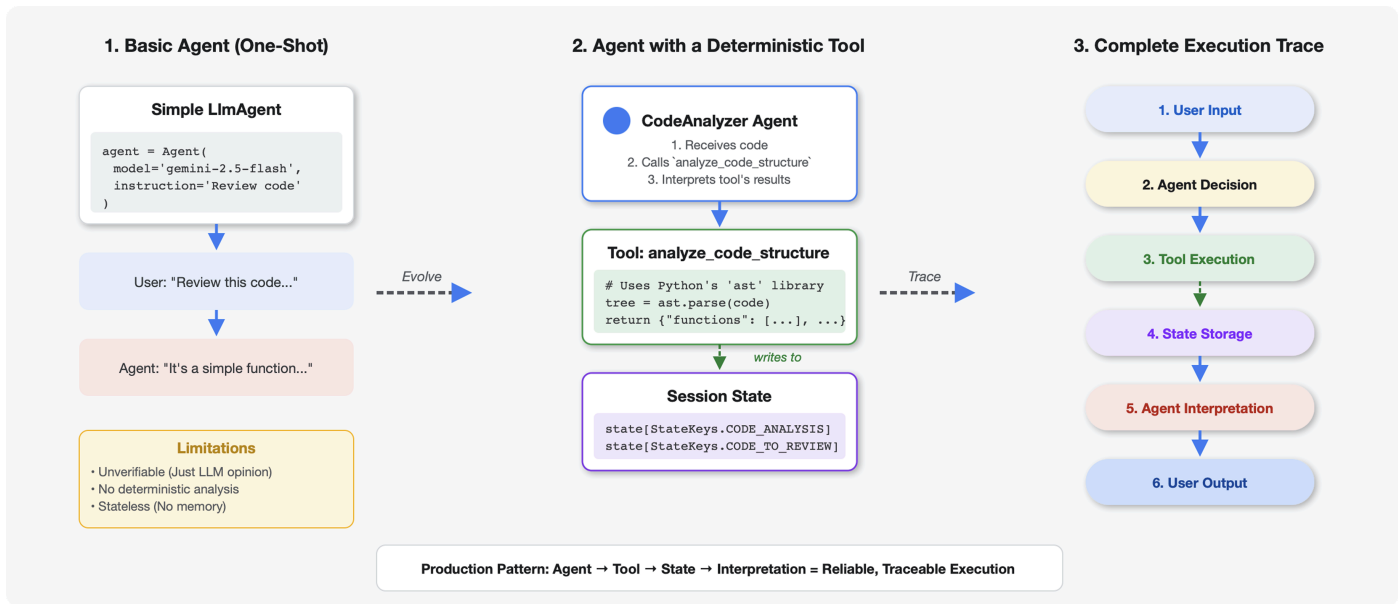
學習成果

現在正式版工作區已準備就緒：

- ✓ 已設定及驗證 Google Cloud 雲端專案
- ✓ 已測試基本代理程式，瞭解限制
- ✓ 已準備好包含策略性預留位置的專案程式碼
- ✓ 已在虛擬環境中隔離依附元件
- ✓ 已啟用所有必要 API
- ✓ 已準備好用於部署的 Container Registry
- ✓ 已正確設定 IAM 權限
- ✓ 已正確設定環境變數

現在您已準備好運用確定性工具、狀態管理和適當架構，建構真正的 AI 系統。

4. 建立第一個代理程式



工具與 LLM 的不同之處

當您詢問 LLM 「這段程式碼中有多少函式？」時，LLM 會使用模式比對和估算。使用呼叫 Python `ast.parse()` 的工具時，系統會剖析實際的語法樹狀結構，不會進行任何猜測，每次都會得到相同結果。

本節將建構工具，以決定性方式分析程式碼結構，然後將其連結至知道何時叫用該工具的代理程式。

步驟 1：瞭解 Scaffold

現在來看看您要填寫的結構。

👉 開啟

`code_review_assistant/tools.py`

您會看到 `analyze_code_structure` 函式，其中包含標示程式碼新增位置的預留位置註解。函式已具備基本結構，您將逐步強化。

步驟 2：新增狀態儲存空間

狀態儲存空間可讓管道中的其他代理程式存取工具結果，不必重新執行分析。

👉 尋找：

```
# MODULE_4_STEP_2_ADD_STATE_STORAGE
```

👉 將該單行程式碼替換為：

```
# Store code and analysis for other agents to access
tool_context.state[StateKeys.CODE_TO_REVIEW] = code
tool_context.state[StateKeys.CODE_ANALYSIS] = analysis
tool_context.state[StateKeys.CODE_LINE_COUNT] = len(code.splitlines())
```

傳回值與狀態：兩者有何差異？

- **傳回值**：LLM 立即看到的內容。用於狀態訊息、摘要和 LLM 目前需要的資訊。
- **狀態儲存空間**：其他代理程式/工具稍後可以讀取的內容。用於需要透過管道保留的詳細資料。

您可以將傳回值視為「大聲說出」，而狀態則視為「在所有服務專員都能看到的共用白板上書寫」。

為什麼要使用 `StateKeys` 常數？

 **無障礙功能注意事項**：如果難以閱讀這些陰影區塊中的程式碼，請使用程式碼區塊右上角的淺色/深色模式切換按鈕，切換至淺色模式。

請注意，我們使用 `StateKeys.CODE_TO_REVIEW`，而非字串 `"code_to_review"`：

```
# Without constants - prone to typos
tool_context.state["code_to_review"] = code
tool_context.state["code_to_reveiw"] # Typo! Returns None silently

# With constants - typos caught by IDE
tool_context.state[StateKeys.CODE_TO_REVIEW] = code
tool_context.state[StateKeys.CODE_TO_REVEIW] # Error immediately!
```

常數定義於 `code_review_assistant/constants.py`：

```
class StateKeys:
    CODE_TO_REVIEW = "code_to_review"
    CODE_ANALYSIS = "code_analysis"
    # ... more keys
```

這樣一來，就能避免只會在正式環境中出現的錯誤。如果多個代理程式共用狀態 (如第 5 模組所示)，一個錯字就會導致整個管道無聲無息地中斷。常數可避免打錯字，IDE 會立即偵測到錯誤。

步驟 3：使用執行緒集區新增非同步剖析功能

我們的工具需要剖析 AST，但不能封鎖其他作業。讓我們使用執行緒集區新增非同步執行作業。

 **尋找：**

```
# MODULE_4_STEP_3_ADD_ASYNC
```

 **將該單行程式碼替換為：**

```
# Parse in thread pool to avoid blocking the event loop
loop = asyncio.get_event_loop()
with ThreadPoolExecutor() as executor:
    tree = await loop.run_in_executor(executor, ast.parse, code)
```

讓工具不會造成阻斷

這個模式可防止工具凍結其他作業。各部分的用途如下：

`async def` 函式簽章 (已在架構中)：

- 允許這項工具使用 `await`
- 讓 ADK 同時執行多個工具
- 建構效能優異的非封鎖代理程式時，這項功能不可或缺。雖然 ADK 框架可以包裝標準同步函式，但該工具會阻斷所有其他並行作業，直到完成為止。對於可投入正式環境的代理程式，`async def` 是標準。

`run_in_executor` 模式 (您剛才新增的項目)：

```
loop = asyncio.get_event_loop()
with ThreadPoolExecutor() as executor:
    tree = await loop.run_in_executor(executor, ast.parse, code)
```

- 在獨立執行緒中執行耗用大量 CPU 資源的 `ast.parse`
- `await` 會暫停這項工具，同時執行緒會繼續運作
- 暫停期間可以執行其他工具
- 防止事件迴圈遭到封鎖

為什麼這兩種都是必要的：

```
# Just async def - still blocks everything!
async def my_tool():
    tree = ast.parse(code) # Blocks for 100ms, nothing else runs

# With thread pool - work happens in background
async def my_tool():
    tree = await loop.run_in_executor(executor, ast.parse, code)
    # Other tools run while ast.parse works in the thread
```

這是 ADK 建議的 CPU 密集型作業模式，詳情請參閱效能[指南](#)。

步驟 4：擷取完整資訊

現在，讓我們擷取類別、匯入項目和詳細指標，也就是完成程式碼審查所需的一切。

👉 尋找：

```
# MODULE_4_STEP_4_EXTRACT_DETAILS
```

👉 將該單行程式碼替換為：

```
# Extract comprehensive structural information
analysis = await loop.run_in_executor(
    executor, _extract_code_structure, tree, code
)
```

👉 驗證：函式

`analyze_code_structure`

in

`tools.py`

的中央主體如下所示：

```
# Parse in thread pool to avoid blocking the event loop
loop = asyncio.get_event_loop()
with ThreadPoolExecutor() as executor:
    tree = await loop.run_in_executor(executor, ast.parse, code)

# Extract comprehensive structural information
analysis = await loop.run_in_executor(
    executor, _extract_code_structure, tree, code
)

# Store code and analysis for other agents to access
tool_context.state[StateKeys.CODE_TO_REVIEW] = code
tool_context.state[StateKeys.CODE_ANALYSIS] = analysis
tool_context.state[StateKeys.CODE_LINE_COUNT] = len(code.splitlines())
```

👉 現在請捲動至

`tools.py`

並找到：

```
# MODULE_4_STEP_4_HELPER_FUNCTION
```

👉 將該單行程式碼替換為完整的輔助函式：

```

def _extract_code_structure(tree: ast.AST, code: str) -> Dict[str, Any]:
    """
    Helper function to extract structural information from AST.
    Runs in thread pool for CPU-bound work.
    """

    functions = []
    classes = []
    imports = []
    docstrings = []

    for node in ast.walk(tree):
        if isinstance(node, ast.FunctionDef):
            func_info = {
                'name': node.name,
                'args': [arg.arg for arg in node.args.args],
                'lineno': node.lineno,
                'has_docstring': ast.get_docstring(node) is not None,
                'is_async': isinstance(node, ast.AsyncFunctionDef),
                'decorators': [d.id for d in node.decorator_list
                              if isinstance(d, ast.Name)]
            }
            functions.append(func_info)

            if func_info['has_docstring']:
                docstrings.append(f"{node.name}: {ast.get_docstring(node)[:50]}...")

        elif isinstance(node, ast.ClassDef):
            methods = []
            for item in node.body:
                if isinstance(item, ast.FunctionDef):
                    methods.append(item.name)

            class_info = {
                'name': node.name,
                'lineno': node.lineno,
                'methods': methods,
                'has_docstring': ast.get_docstring(node) is not None,
                'base_classes': [base.id for base in node.bases
                                if isinstance(base, ast.Name)]
            }
            classes.append(class_info)

        elif isinstance(node, ast.Import):
            for alias in node.names:
                imports.append({
                    'module': alias.name,
                    'alias': alias.asname,
                    'type': 'import'
                })

        elif isinstance(node, ast.ImportFrom):
            imports.append({
                'module': node.module or '',

```

```

        'names': [alias.name for alias in node.names],
        'type': 'from_import',
        'level': node.level
    })

return {
    'functions': functions,
    'classes': classes,
    'imports': imports,
    'docstrings': docstrings,
    'metrics': {
        'line_count': len(code.splitlines()),
        'function_count': len(functions),
        'class_count': len(classes),
        'import_count': len(imports),
        'has_main': any(f['name'] == 'main' for f in functions),
        'has_if_main': '__main__' in code,
        'avg_function_length': _calculate_avg_function_length(tree)
    }
}

def _calculate_avg_function_length(tree: ast.AST) -> float:
    """Calculate average function length in lines."""
    function_lengths = []

    for node in ast.walk(tree):
        if isinstance(node, ast.FunctionDef):
            if hasattr(node, 'end_lineno') and hasattr(node, 'lineno'):
                length = node.end_lineno - node.lineno + 1
                function_lengths.append(length)

    if function_lengths:
        return sum(function_lengths) / len(function_lengths)
    return 0.0

```

為什麼要使用輔助函式？

`_extract_code_structure` 輔助程式為：

- 同步 (無 `async`) - 必須在 `run_in_executor` 中執行
- **CPU 繫結** - 在執行緒中執行實際的剖析工作
- 可測試：您可以與完整工具分開測試
- 可重複使用 - 其他工具可能可以使用

主要工具函式 (`analyze_code_structure`) 會處理非同步執行、狀態管理和錯誤處理。這個小幫手只會從 AST 擷取資訊。

我們會擷取哪些資訊？

我們會擷取每個函式的下列資訊：

- 名稱和引數 (適用於說明文件)
- 行號 (用於回報錯誤)
- 是否有說明字串 (樣式檢查)
- 修飾符 (偵測特殊模式)

對於類別，我們會擷取：

- 類別中定義的方法
- 基礎類別 (繼承分析)
- 文件狀態

這項詳細分析可讓樣式檢查工具、測試執行器和意見回饋合成器提供具體可行的洞察資訊。

步驟 5：與服務專員連線

現在，我們將工具連結至代理程式，讓代理程式知道何時使用工具，以及如何解讀結果。

👉 開啟

```
code_review_assistant/sub_agents/review_pipeline/code_analyzer.py
```

👉 尋找：

```
# MODULE_4_STEP_5_CREATE_AGENT
```

👉 將該單行替換為完整的正式版代理程式：

```
code_analyzer_agent = Agent(
    name="CodeAnalyzer",
    model=config.worker_model,
    description="Analyzes Python code structure and identifies components",
    instruction="""You are a code analysis specialist responsible for understanding code
structure.
```

Your task:

1. Take the code submitted by the user (it will be provided in the user message)
2. Use the analyze_code_structure tool to parse and analyze it
3. Pass the EXACT code to your tool – do not modify, fix, or "improve" it
4. Identify all functions, classes, imports, and structural patterns
5. Note any syntax errors or structural issues
6. Store the analysis in state for other agents to use

CRITICAL:

- Pass the code EXACTLY as provided to the analyze_code_structure tool
- Do not fix syntax errors, even if obvious
- Do not add missing imports or fix indentation
- The goal is to analyze what IS there, not what SHOULD be there

When calling the tool, pass the code as a string to the 'code' parameter.

If the analysis fails due to syntax errors, clearly report the error location and type.

Provide a clear summary including:

- Number of functions and classes found
- Key structural observations
- Any syntax errors or issues detected
- Overall code organization assessment""",
tools=[FunctionTool(func=analyze_code_structure)],
output_key="structure_analysis_summary"
)

為什麼需要這麼詳細的指示？

這項指示強調「確切代碼」和「請勿修正」，原因如下：

- 大型語言模型自然會想提供協助，並修正明顯錯誤
- 但這是「審查」代理，不是「修正」代理，兩者關注的重點不同
- 我們希望對實際情況進行誠實分析
- 修正管道 (模組 6) 會處理修正內容

編號步驟可清楚引導 LLM 完成工作流程，並減少何時該執行哪些動作的模糊空間。

常見的代理程式錯誤：過度熱心的 LLM

如果沒有「EXACT code」指令，就會得到誤導性結果：

```
# User submits:
def add(a,b):return a+b # Missing spaces, wrong style

# Without instruction, LLM "helpfully" calls tool with:
def add(a, b):
    return a + b

# Style checker analyzes the "fixed" code
# Reports: "Perfect! No issues found!"
# User gets completely wrong feedback
```

明確的指令會告訴 LLM：你的工作是分析，而不是改進，因此可避免這種情況。請如實轉達您收到的內容。

瞭解 output_key

`output_key="structure_analysis_summary"` 的意義：

- 這個代理程式說的內容都會儲存在 `state["structure_analysis_summary"]`
- 管道中的後續服務專員可以讀取這份摘要
- 無須重新執行分析工具
- 效率高：分析一次，多次參照

在第 5 個單元中，我們將建構樣式檢查器和意見回饋合成器，屆時會參考這份摘要。

測試程式碼分析器

現在請確認分析器是否正常運作。

👉 執行測試指令碼：

```
python tests/test_code_analyzer.py
```

測試腳本會使用 `python-dotenv` 從 `.env` 檔案自動載入設定，因此不需要手動設定環境變數。

預期的輸出內容：

```
INFO:code_review_assistant.config:Code Review Assistant Configuration Loaded:
INFO:code_review_assistant.config: - GCP Project: your-project-id
INFO:code_review_assistant.config: - Artifact Bucket: gs://your-project-artifacts
INFO:code_review_assistant.config: - Models: worker=gemini-2.5-flash, critic=gemini-2.5-pro
Testing code analyzer...
INFO:code_review_assistant.tools:Tool: Analysis complete - 2 functions, 1 classes

=== Analyzer Response ===
The analysis of the provided code shows the following:

* **Functions Found:** 2
  * `add(a, b)` : A global function at line 2.
  * `multiply(self, x, y)` : A method within the `Calculator` class.

* **Classes Found:** 1
  * `Calculator` : A class defined at line 5. Contains one method, `multiply`.

* **Imports:** 0

* **Structural Patterns:** The code defines one global function and one class
  with a single method. Both are simple, each with a single return statement.

* **Syntax Errors/Issues:** No syntax errors detected.

* **Overall Code Organization:** The code is well-organized for its small size,
  clearly defining a function and a class with a method.
```

剛才發生了什麼事：

1. 測試腳本已自動載入 `.env` 設定
2. 您的 `analyze_code_structure()` 工具使用 Python 的 AST 剖析程式碼
3. `_extract_code_structure()` 輔助程式會擷取函式、類別和指標
4. 結果會使用 `StateKeys` 常數儲存在工作階段狀態中
5. 程式碼分析器代理程式解讀結果並提供摘要

疑難排解：

- 「No module named 'code_review_assistant'」(找不到名為「code_review_assistant」的模組)：從專案根目錄執行 `pip install -e .`
- 「缺少金鑰輸入引數」：確認 `.env` 含有 `GOOGLE_CLOUD_PROJECT`、`GOOGLE_CLOUD_LOCATION` 和 `GOOGLE_GENAI_USE_VERTEXAI=true`

建構項目

您現在擁有可直接用於正式環境的程式碼分析器，可執行下列操作：

- ✅ 剖析實際的 Python AST - 具決定性，而非模式比對
- ✅ 將結果儲存在狀態中 - 其他代理程式可以存取分析結果
- ✅ 非同步執行 - 不會封鎖其他工具
- ✅ 擷取完整資訊 - 函式、類別、匯入項目、指標

- ✔ 妥善處理錯誤 - 報告語法錯誤和行號
- ✔ 連結至代理程式 - LLM 知道何時及如何使用

已掌握的關鍵概念

工具與代理程式：

- 工具會執行確定性工作 (AST 剖析)
- 代理會決定何時使用工具並解讀結果

傳回值與狀態：

- 回覆：LLM 立即看到的內容
- 狀態：其他代理會保留哪些內容

狀態鍵常數：

- 防止多代理系統發生錯別字
- 做為代理程式之間的合約
- 代理人分享資料時至關重要

非同步 + 執行緒集區：

- `async def` 可讓工具暫停執行
- 執行緒集區會在背景執行 CPU 繫結工作
- 兩者共同確保事件迴圈保持回應狀態

輔助函式：

- 將同步輔助程式與非同步工具分開
- 讓程式碼可供測試及重複使用

服務專員指示：

- 詳細指令可避免常見的 LLM 錯誤
- 明確說明「不要」做什麼 (不要修正程式碼)
- 清除工作流程步驟，確保一致性

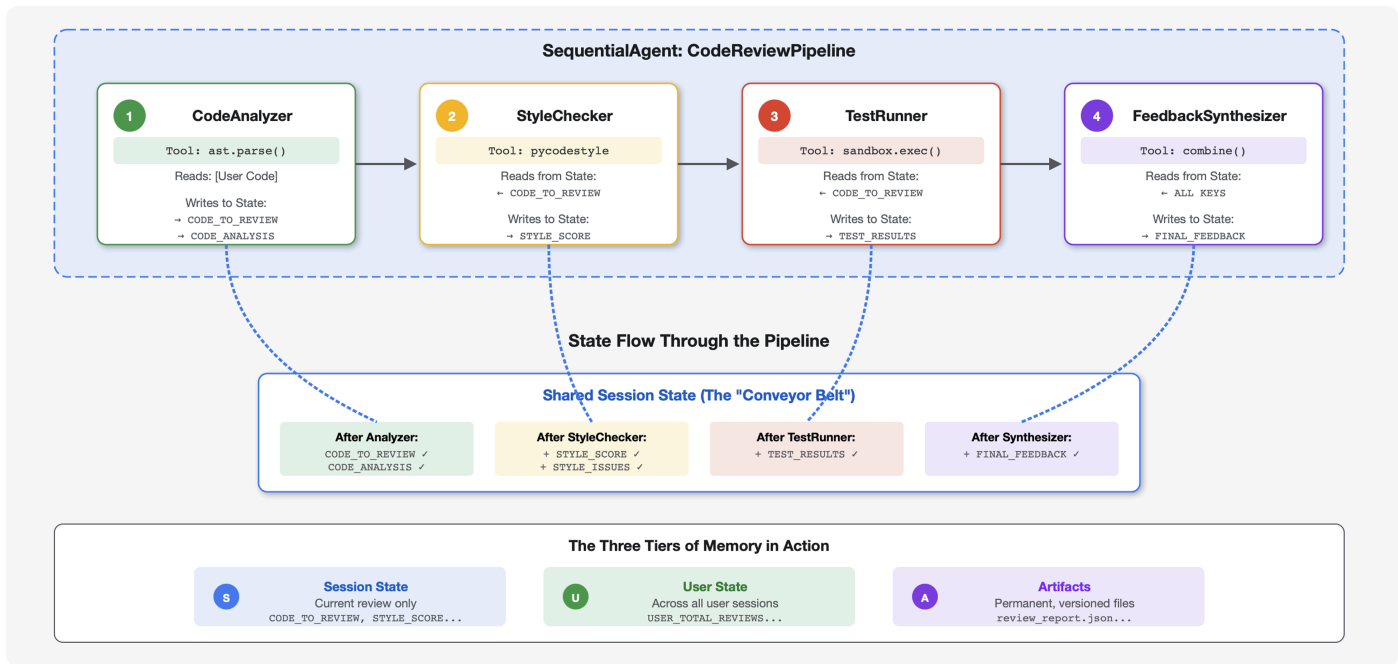
後續步驟

在單元 5 中，您將新增：

- 樣式檢查工具，可從狀態讀取程式碼
- 實際執行測試的測試執行工具
- 意見回饋統整器，可整合所有分析結果

您會瞭解狀態如何透過循序管道流動，以及多個代理程式讀取及寫入相同資料時，常數模式為何重要。

5. 建構管道：多個代理協同運作



簡介

在單元 4 中，您建立了一個可分析程式碼結構的代理。但要進行全面的程式碼審查，不只是剖析程式碼即可，還需要檢查樣式、執行測試，以及綜合分析智慧回饋。

這個模組會建構由 **4 個代理組成的管道**，這些代理會依序協作，各自提供專業分析：

1. 程式碼分析工具 (來自第 4 堂課) - 剖析結構
2. 樣式檢查工具：找出違反樣式的內容
3. 測試執行工具 - 執行及驗證測試
4. 意見回饋綜合分析器 - 將所有內容整合為實用意見回饋

重要概念：狀態即通訊管道。每個代理程式都會讀取先前代理程式寫入狀態的內容，加入自己的分析，然後將經過擴充的狀態傳遞給下一個代理程式。如果多個代理共用資料，單元 4 的常數模式就非常重要。

您將建構的內容預覽：提交雜亂的程式碼 → 觀察狀態流經 4 個代理程式 → 根據過去的模式接收包含個人化意見回饋的完整報告。

步驟 1：新增樣式檢查工具 + 代理程式

樣式檢查工具會使用 `pycodestyle` 找出違反 PEP 8 的情況，這是一種確定性 Linter，而非以 LLM 為基礎的解讀方式。

新增樣式檢查工具

👉 開啟

```
code_review_assistant/tools.py
```

👉 尋找：

```
# MODULE_5_STEP_1_STYLE_CHECKER_TOOL
```

👉 將該單行程式碼替換為：

```

async def check_code_style(code: str, tool_context: ToolContext) -> Dict[str, Any]:
    """
    Checks code style compliance using pycodestyle (PEP 8).

    Args:
        code: Python source code to check (or will retrieve from state)
        tool_context: ADK tool context

    Returns:
        Dictionary containing style score and issues
    """
    logger.info("Tool: Checking code style...")

    try:
        # Retrieve code from state if not provided
        if not code:
            code = tool_context.state.get(StateKeys.CODE_TO_REVIEW, '')
            if not code:
                return {
                    "status": "error",
                    "message": "No code provided or found in state"
                }

        # Run style check in thread pool
        loop = asyncio.get_event_loop()
        with ThreadPoolExecutor() as executor:
            result = await loop.run_in_executor(
                executor, _perform_style_check, code
            )

        # Store results in state
        tool_context.state[StateKeys.STYLE_SCORE] = result['score']
        tool_context.state[StateKeys.STYLE_ISSUES] = result['issues']
        tool_context.state[StateKeys.STYLE_ISSUE_COUNT] = result['issue_count']

        logger.info(f"Tool: Style check complete - Score: {result['score']}/100, "
                    f"Issues: {result['issue_count']}")

        return result

    except Exception as e:
        error_msg = f"Style check failed: {str(e)}"
        logger.error(f"Tool: {error_msg}", exc_info=True)

        # Set default values on error
        tool_context.state[StateKeys.STYLE_SCORE] = 0
        tool_context.state[StateKeys.STYLE_ISSUES] = []

        return {
            "status": "error",
            "message": error_msg,

```



```
    "score": 0  
  }
```

👉 現在請捲動至檔案結尾，然後找出：

```
# MODULE_5_STEP_1_STYLE_HELPERS
```

👉 將該單行取代為輔助函式：

```

def _perform_style_check(code: str) -> Dict[str, Any]:
    """Helper to perform style check in thread pool."""
    import io
    import sys

    with tempfile.NamedTemporaryFile(mode='w', suffix='.py', delete=False) as tmp:
        tmp.write(code)
        tmp_path = tmp.name

    try:
        # Capture stdout to get pycodestyle output
        old_stdout = sys.stdout
        sys.stdout = captured_output = io.StringIO()

        style_guide = pycodestyle.StyleGuide(
            quiet=False, # We want output
            max_line_length=100,
            ignore=['E501', 'W503']
        )

        result = style_guide.check_files([tmp_path])

        # Restore stdout
        sys.stdout = old_stdout

        # Parse captured output
        output = captured_output.getvalue()
        issues = []

        for line in output.strip().split('\n'):
            if line and ':' in line:
                parts = line.split(':', 4)
                if len(parts) >= 4:
                    try:
                        issues.append({
                            'line': int(parts[1]),
                            'column': int(parts[2]),
                            'code': parts[3].split()[0] if len(parts) > 3 else 'E000',
                            'message': parts[3].strip() if len(parts) > 3 else 'Unknown
error'

                        })
                    except (ValueError, IndexError):
                        pass

        # Add naming convention checks
        try:
            tree = ast.parse(code)
            naming_issues = _check_naming_conventions(tree)
            issues.extend(naming_issues)
        except SyntaxError:
            pass # Syntax errors will be caught elsewhere

```

```

    # Calculate weighted score
    score = _calculate_style_score(issues)

    return {
        "status": "success",
        "score": score,
        "issue_count": len(issues),
        "issues": issues[:10], # First 10 issues
        "summary": f"Style score: {score}/100 with {len(issues)} violations"
    }

finally:
    if os.path.exists(tmp_path):
        os.unlink(tmp_path)

def _check_naming_conventions(tree: ast.AST) -> List[Dict[str, Any]]:
    """Check PEP 8 naming conventions."""
    naming_issues = []

    for node in ast.walk(tree):
        if isinstance(node, ast.FunctionDef):
            # Skip private/protected methods and __main__
            if not node.name.startswith('_') and node.name != node.name.lower():
                naming_issues.append({
                    'line': node.lineno,
                    'column': node.col_offset,
                    'code': 'N802',
                    'message': f"N802 function name '{node.name}' should be lowercase"
                })
        elif isinstance(node, ast.ClassDef):
            # Check if class name follows CapWords convention
            if not node.name[0].isupper() or '_' in node.name:
                naming_issues.append({
                    'line': node.lineno,
                    'column': node.col_offset,
                    'code': 'N801',
                    'message': f"N801 class name '{node.name}' should use CapWords
convention"
                })

    return naming_issues

def _calculate_style_score(issues: List[Dict[str, Any]]) -> int:
    """Calculate weighted style score based on violation severity."""
    if not issues:
        return 100

    # Define weights by error type
    weights = {
        'E1': 10, # Indentation errors

```

```

'E2': 3, # Whitespace errors
'E3': 5, # Blank line errors
'E4': 8, # Import errors
'E5': 5, # Line length
'E7': 7, # Statement errors
'E9': 10, # Syntax errors
'W2': 2, # Whitespace warnings
'W3': 2, # Blank line warnings
'W5': 3, # Line break warnings
'N8': 7, # Naming conventions
}

total_deduction = 0
for issue in issues:
    code_prefix = issue['code'][:2] if len(issue['code']) >= 2 else 'E2'
    weight = weights.get(code_prefix, 3)
    total_deduction += weight

# Cap at 100 points deduction
return max(0, 100 - min(total_deduction, 100))

```

正式版模式：分離輔助函式

請注意以下結構：

- **主要工具** (`check_code_style`)：非同步，處理狀態、錯誤處理
- **輔助程式** (`_perform_style_check`)：同步、純邏輯，在執行緒集區中執行
- **子輔助程式** (`_check_naming_conventions` 、 `_calculate_style_score`)：專注於實用功能

這種分離方式可提供以下優勢：

- **可測試性**：獨立測試輔助程式
- **可重複使用**：其他工具可使用相同邏輯
- **執行緒安全**：同步輔助程式可在執行緒集區中運作
- **可維護性**：每個函式都只有單一責任

加權評分系統 (`_calculate_style_score`) 會優先處理嚴重違規事項 (縮排、語法)，而非次要違規事項 (空白字元)，因此評估品質時比單純計算更準確。

狀態擷取模式

工具會檢查是否已提供代碼，如果沒有，則會從狀態中擷取代碼：

```
if not code:
    code = tool_context.state.get(StateKeys.CODE_TO_REVIEW, '')
```

這項工具的彈性在於：

- **管道使用**：代理會自動從狀態讀取
- **獨立使用**：可直接傳遞程式碼進行測試
- **錯誤處理**：先驗證代碼是否存在，再進行處理

這個模式會出現在整個製作工具中，請務必一律提供狀態的回退機制。

新增樣式檢查代理程式

👉 開啟

```
code_review_assistant/sub_agents/review_pipeline/style_checker.py
```

👉 尋找：

```
# MODULE_5_STEP_1_INSTRUCTION_PROVIDER
```

👉 將該單行程式碼替換為：

```

async def style_checker_instruction_provider(context: ReadonlyContext) -> str:
    """Dynamic instruction provider that injects state variables."""
    template = """You are a code style expert focused on PEP 8 compliance.

Your task:
1. Use the check_code_style tool to validate PEP 8 compliance
2. The tool will retrieve the ORIGINAL code from state automatically
3. Report violations exactly as found
4. Present the results clearly and confidently

CRITICAL:
- The tool checks the code EXACTLY as provided by the user
- Do not suggest the code was modified or fixed
- Report actual violations found in the original code
- If there are style issues, they should be reported honestly

Call the check_code_style tool with an empty string for the code parameter,
as the tool will retrieve the code from state automatically.

When presenting results based on what the tool returns:
- State the exact score from the tool results
- If score >= 90: "Excellent style compliance!"
- If score 70-89: "Good style with minor improvements needed"
- If score 50-69: "Style needs attention"
- If score < 50: "Significant style improvements needed"

List the specific violations found (the tool will provide these):
- Show line numbers, error codes, and messages
- Focus on the top 10 most important issues

Previous analysis: {structure_analysis_summary}

Format your response as:
## Style Analysis Results
- Style Score: [exact score]/100
- Total Issues: [count]
- Assessment: [your assessment based on score]

## Top Style Issues
[List issues with line numbers and descriptions]

## Recommendations
[Specific fixes for the most critical issues]"""

    return await instructions_utils.inject_session_state(template, context)

```

👉 尋找：

```
# MODULE_5_STEP_1_STYLE_CHECKER_AGENT
```

👉 將該單行程式碼替換為：

```
style_checker_agent = Agent(
    name="StyleChecker",
    model=config.worker_model,
    description="Checks Python code style against PEP 8 guidelines",
    instruction=style_checker_instruction_provider,
    tools=[FunctionTool(func=check_code_style)],
    output_key="style_check_summary"
)
```

動態指令供應商

請注意模式：

```
async def style_checker_instruction_provider(context: ReadonlyContext) -> str:
    template = "......"
    return await instructions_utils.inject_session_state(template, context)
```

為什麼要使用動態指令而非靜態指令？

靜態 (可能符合您的預期)：

```
instruction="Check the code style and report issues"
```

問題：一般問題，沒有先前服務專員發現問題的背景資訊

動態 (製作模式)：

```
instruction=style_checker_instruction_provider
```

- 每次叫用都會執行 - 取得最新狀態
- 從狀態插入 `{structure_analysis_summary}` 等值
- 根據目前的評論內容調整指示
- LLM 會看到這段特定程式碼的具體資料

`instructions_utils.inject_session_state` 呼叫會將 `{key_name}` 預留位置替換為 `context.state` 中的實際值。

步驟 2：新增 Test Runner Agent

測試執行器會產生全面測試，並使用內建的程式碼執行器執行測試。

👉 開啟

```
code_review_assistant/sub_agents/review_pipeline/test_runner.py
```

👉 尋找：

```
# MODULE_5_STEP_2_INSTRUCTION_PROVIDER
```

👉 將該單行程式碼替換為：

```
async def test_runner_instruction_provider(context: ReadonlyContext) -> str:
    """Dynamic instruction provider that injects the code_to_review directly."""
    template = """You are a testing specialist who creates and runs tests for Python code.

THE CODE TO TEST IS:
{code_to_review}

YOUR TASK:
1. Understand what the function appears to do based on its name and structure
2. Generate comprehensive tests (15-20 test cases)
3. Execute the tests using your code executor
4. Analyze results to identify bugs vs expected behavior
5. Output a detailed JSON analysis

TESTING METHODOLOGY:
- Test with the most natural interpretation first
- When something fails, determine if it's a bug or unusual design
- Test edge cases, boundaries, and error scenarios
- Document any surprising behavior

Execute your tests and output ONLY valid JSON with this structure:
- "test_summary": object with "total_tests_run", "tests_passed", "tests_failed",
  "tests_with_errors", "critical_issues_found"
- "critical_issues": array of objects, each with "type", "description", "example_input",
  "expected_behavior", "actual_behavior", "severity"
- "test_categories": object with "basic_functionality", "edge_cases", "error_handling" (each
  containing "passed", "failed", "errors" counts)
- "function_behavior": object with "apparent_purpose", "actual_interface",
  "unexpected_requirements"
- "verdict": object with "status" (WORKING/BUGGY/BROKEN), "confidence" (high/medium/low),
  "recommendation"

Do NOT output the test code itself, only the JSON analysis."""

    return await instructions_utils.inject_session_state(template, context)
```

👉 尋找：

```
# MODULE_5_STEP_2_TEST_RUNNER_AGENT
```

👉 將該單行程式碼替換為：


```
test_runner_agent = Agent(
    name="TestRunner",
    model=config.critic_model,
    description="Generates and runs tests for Python code using safe code execution",
    instruction=test_runner_instruction_provider,
    code_executor=BuiltInCodeExecutor(),
    output_key="test_execution_summary"
)
```

為什麼要使用評論家模型進行測試？

請注意，使用 `model=config.critic_model` 而非 `worker_model`：

```
test_runner_agent = Agent(
    model=config.critic_model, # More capable model
    ...
)
```

選擇 Worker 模型或 Critic 模型：

- 工作人員 (`gemini-2.5-flash`)：速度快、價格便宜，適合處理機械式工作
- 專家 (`gemini-2.5-pro`)：速度較慢、費用較高，但推論能力較佳

測試必須符合下列條件：

- 從名稱/結構瞭解函式意圖
- 生成 15 到 20 個有意義的測試案例
- 區分錯誤與設計選擇
- 分析失敗模式

這種程度的推論能力足以證明使用功能更強大 (且更昂貴) 的模型是合理的。分析器和樣式檢查器使用工作站模型，因為這類模型較為機械化。

程式碼執行的強大功能

`BuiltInCodeExecutor` 是區別程式碼審查員與只會談論程式碼的 AI 的關鍵。`TestRunner` 代理程式產生測試案例時，會在安全的 Python 沙箱中執行這些案例。也就是說：

- 實際驗證：實際執行測試，找出靜態分析遺漏的執行階段錯誤
- 有憑有據，而非臆測：如果系統顯示「第 4 行發生 `TypeError`」，表示系統執行程式碼時發現錯誤
- 結果一致：每次執行相同測試都會產生相同結果
- 安全執行：沙箱與系統隔離

這個內建執行器非常適合我們的用途，也就是測試純演算法程式碼，例如資料結構、排序演算法和運算邏輯。

瞭解沙箱限制

`BuiltInCodeExecutor` 沙箱設有以下限制：

- 未安裝套件：無法使用 `pip install`
- 無法存取網路：無法發出 HTTP 要求或存取 API
- 執行時間有限：長時間執行的程式碼會逾時

這些並非瑕疵，而是安全防護功能。將沙箱限制為純 Python，即可安全執行不受信任的程式碼。

正式環境替代方案：如要在 GKE 上部署正式環境，請考慮使用 `GkeCodeExecutor`，在具有 gVisor 的獨立 Kubernetes Pod 中執行程式碼。

步驟 3：瞭解跨工作階段學習的記憶體

建構意見回饋合成器之前，您需要瞭解「狀態」和「記憶體」之間的差異，這兩種儲存機制用途不同。

狀態與記憶體：主要區別

我們以程式碼審查的具體範例來說明：

狀態 (僅限目前工作階段)：

```
# Data from THIS review session
tool_context.state[StateKeys.STYLE_ISSUES] = [
    {"line": 5, "code": "E231", "message": "missing whitespace"},
    {"line": 12, "code": "E701", "message": "multiple statements"}
]
```

- 範圍：僅限這個對話
- 用途：在目前管道中的代理之間傳遞資料
- 居住在：`Session` 物件
- 生命週期：工作階段結束時捨棄

記憶體 (所有過去的工作階段)：

```
# Learned from 50 previous reviews
"User frequently forgets docstrings on helper functions"
"User tends to write long functions (avg 45 lines)"
"User improved error handling after feedback in session #23"
```

- 範圍：這位使用者的所有過往工作階段
- 用途：瞭解模式、提供個人化意見回饋
- 居住地：`MemoryService`
- 生命週期：在不同工作階段之間持續存在，可供搜尋

為什麼需要同時提供這兩種意見回饋：

假設合成器會產生意見回饋：

僅使用「州」(目前審查)：

```
"Function `calculate_total` has no docstring."
```

一般、機械式的回饋。

使用狀態 + 記憶體 (目前和過去的模式)：

```
"Function `calculate_total` has no docstring. This is the 4th review  
where helper functions lacked documentation. Consider adding docstrings  
as you write functions, not afterwards - you mentioned in our last  
session that you find it easier that way."
```

隨著時間推移，個人化、情境式參考資料會越來越準確。

如要部署至正式環境，您有[多種選項](#)：

選項 1：VertexAiMemoryBankService (進階)

- **功能**：透過 LLM 從對話中擷取有意義的事實
- **搜尋**：語意搜尋 (理解意義，而不只是關鍵字)
- **記憶體管理**：隨著時間推移，自動整合及更新記憶體
- **必要條件**：Google Cloud 專案 + Agent Engine 設定
- **適用情況**：您希望回憶錄能不斷演進，並提供精緻的個人化體驗
- **範例**：「使用者偏好函式程式設計」(從 10 則有關程式碼樣式的對話中擷取)

選項 2：繼續使用 InMemoryMemoryService + 持續性工作階段

- **用途**：儲存關鍵字搜尋的完整對話記錄
- **搜尋**：在過去的對話中比對基本關鍵字
- **記憶體管理**：你可以控管要儲存的内容 (透過 `add_session_to_memory`)
- **必要條件**：僅限持續性 `SessionService` (例如 `VertexAiSessionService` 或 `DatabaseSessionService`)
- **適用於**：您需要簡單搜尋過往對話，且不需要 LLM 處理
- **範例**：搜尋「docstring」會傳回所有提及該字詞的對話

瞭解服務關係

您可以這樣理解：

SessionService (管理對話)：

- 商店：事件、目前對話的狀態
- 例如：VertexAiSessionService、DatabaseSessionService、InMemorySessionService
- 適用於：保留目前的對話

MemoryService (跨工作階段知識)：

- 儲存：先前對話中的資訊
- 範例：
 - InMemoryMemoryService：儲存完整記錄、關鍵字搜尋
 - VertexAiMemoryBankService：擷取知識、語意搜尋
- 用途：從過去的工作階段擷取內容

搭配運作方式：

```
# After code review completes
session = await session_service.get_session(...)

# Add session to memory for future reference
await memory_service.add_session_to_memory(session)

# Future reviews can search memory
results = await memory_service.search_memory("docstring patterns")
```

您可以使用 VertexAiSessionService，不必使用 Memory Bank (只要保留工作階段即可)，但 Memory Bank 需要工作階段才能擷取資訊。

記憶體如何填入資料

每次完成程式碼審查後：

```
# At the end of a session (typically in your application code)
await memory_service.add_session_to_memory(session)
```

會發生什麼情況：

- InMemoryMemoryService：儲存關鍵字搜尋的完整工作階段事件
- VertexAiMemoryBankService：LLM 會擷取重要事實，並與現有記憶內容合併

日後工作階段可以查詢：

```
# In a tool, search for relevant past feedback
results = tool_context.search_memory("feedback about docstrings")
```

狀態、記憶體和構件：各種用途

現在有三種儲存機制：

州/省：

- 類型：結構化資料 (字典、清單、數字、字串)
- 範例：`{"style_score": 75, "test_pass_rate": 0.8}`
- 使用時機：此管道中的其他代理需要資料
- 存取權：`tool_context.state[StateKeys.STYLE_SCORE]`

記憶體：

- 類型：過去工作階段的可搜尋文字
- 例如：「使用者在 API 呼叫中處理錯誤時遇到困難」
- 適用於：學習模式，以改善日後的訓練課程
- 存取權：`tool_context.search_memory("error handling patterns")`

構件：

- 類型：二進位檔案 (PDF、圖片、Excel 檔案)
- 範例：格式化的最終程式碼審查報告
- 適用於：使用者需要下載/查看檔案
- 存取權：`tool_context.save_artifact("report.pdf", pdf_bytes)`

合成器會使用這三種方法：

- 讀取 **state**，取得目前的分析結果
- 搜尋記憶體中的過往模式
- 儲存最終報表的構件

步驟 4：新增意見回饋綜合工具和代理

意見回饋合成器是管道中最精密的代理程式，這項工具會協調三種工具、使用動態指令，並結合狀態、記憶體和構件。

新增三種合成器工具

👉 開啟

```
code_review_assistant/tools.py
```

👉 尋找：

```
# MODULE_5_STEP_4_SEARCH_PAST_FEEDBACK
```

👉 Replace with Tool 1 - Memory Search (production version)：

```

async def search_past_feedback(developer_id: str, tool_context: ToolContext) -> Dict[str,
Any]:
    """
    Search for past feedback in memory service.

    Args:
        developer_id: ID of the developer (defaults to "default_user")
        tool_context: ADK tool context with potential memory service access

    Returns:
        Dictionary containing feedback search results
    """
    logger.info(f"Tool: Searching for past feedback for developer {developer_id}...")

    try:
        # Default developer ID if not provided
        if not developer_id:
            developer_id = tool_context.state.get(StateKeys.USER_ID, 'default_user')

        # Check if memory service is available
        if hasattr(tool_context, 'search_memory'):
            try:
                # Perform structured searches
                queries = [
                    f"developer:{developer_id} code review feedback",
                    f"developer:{developer_id} common issues",
                    f"developer:{developer_id} improvements"
                ]

                all_feedback = []
                patterns = {
                    'common_issues': [],
                    'improvements': [],
                    'strengths': []
                }

                for query in queries:
                    search_result = await tool_context.search_memory(query)

                    if search_result and hasattr(search_result, 'memories'):
                        for memory in search_result.memories[:5]:
                            memory_text = memory.text if hasattr(memory, 'text') else
str(memory)

                            all_feedback.append(memory_text)

                # Extract patterns
                if 'style' in memory_text.lower():
                    patterns['common_issues'].append('style compliance')
                if 'improved' in memory_text.lower():
                    patterns['improvements'].append('showing improvement')
                if 'excellent' in memory_text.lower():
                    patterns['strengths'].append('consistent quality')

```

```

        # Store in state
        tool_context.state[StateKeys.PAST_FEEDBACK] = all_feedback
        tool_context.state[StateKeys.FEEDBACK_PATTERNS] = patterns

        logger.info(f"Tool: Found {len(all_feedback)} past feedback items")

        return {
            "status": "success",
            "feedback_found": True,
            "count": len(all_feedback),
            "summary": " | ".join(all_feedback[:3]) if all_feedback else "No
feedback",
            "patterns": patterns
        }

    except Exception as e:
        logger.warning(f"Tool: Memory search error: {e}")

    # Fallback: Check state for cached feedback
    cached_feedback = tool_context.state.get(StateKeys.USER_PAST_FEEDBACK_CACHE, [])
    if cached_feedback:
        tool_context.state[StateKeys.PAST_FEEDBACK] = cached_feedback
        return {
            "status": "success",
            "feedback_found": True,
            "count": len(cached_feedback),
            "summary": "Using cached feedback",
            "patterns": {}
        }

    # No feedback found
    tool_context.state[StateKeys.PAST_FEEDBACK] = []
    logger.info("Tool: No past feedback found")

    return {
        "status": "success",
        "feedback_found": False,
        "message": "No past feedback available - this appears to be a first submission",
        "patterns": {}
    }

except Exception as e:
    error_msg = f"Feedback search error: {str(e)}"
    logger.error(f"Tool: {error_msg}", exc_info=True)

    tool_context.state[StateKeys.PAST_FEEDBACK] = []

    return {
        "status": "error",
        "message": error_msg,

```

```
"feedback_found": False
}
```

生產模式：優雅降級

請注意三層級的備援策略：

```
# 1. Try memory service if available
if hasattr(tool_context, 'search_memory'):
    # Search multiple queries, extract patterns

# 2. Fall back to cached feedback in state
cached_feedback = tool_context.state.get(StateKeys.USER_PAST_FEEDBACK_CACHE, [])
if cached_feedback:
    # Use cached data

# 3. Gracefully handle no feedback
return {"feedback_found": False, "message": "...first submission"}
```

這個模式可確保工具絕不會導致管道當機：

- 記憶體服務無法使用？使用快取
- 快取是否為空？Return "no feedback found"
- 一律傳回有效回應，絕不會引發例外狀況

生產工具會優先考量韌性，而非完美。

👉 尋找：

```
# MODULE_5_STEP_4_UPDATE_GRADING_PROGRESS
```

👉 Replace with Tool 2 - Grading Tracker (production version)：


```

async def update_grading_progress(tool_context: ToolContext) -> Dict[str, Any]:
    """
    Updates grading progress counters and metrics in state.
    """
    logger.info("Tool: Updating grading progress...")

    try:
        current_time = datetime.now().isoformat()

        # Build all state changes
        state_updates = {}

        # Temporary (invocation-level) state
        state_updates[StateKeys.TEMP_PROCESSING_TIMESTAMP] = current_time

        # Session-level state
        attempts = tool_context.state.get(StateKeys.GRADING_ATTEMPTS, 0) + 1
        state_updates[StateKeys.GRADING_ATTEMPTS] = attempts
        state_updates[StateKeys.LAST_GRADING_TIME] = current_time

        # User-level persistent state
        lifetime_submissions = tool_context.state.get(StateKeys.USER_TOTAL_SUBMISSIONS, 0) +
1
        state_updates[StateKeys.USER_TOTAL_SUBMISSIONS] = lifetime_submissions
        state_updates[StateKeys.USER_LAST_SUBMISSION_TIME] = current_time

        # Calculate improvement metrics
        current_style_score = tool_context.state.get(StateKeys.STYLE_SCORE, 0)
        last_style_score = tool_context.state.get(StateKeys.USER_LAST_STYLE_SCORE, 0)
        score_improvement = current_style_score - last_style_score

        state_updates[StateKeys.USER_LAST_STYLE_SCORE] = current_style_score
        state_updates[StateKeys.SCORE_IMPROVEMENT] = score_improvement

        # Track test results if available
        test_results = tool_context.state.get(StateKeys.TEST_EXECUTION_SUMMARY, {})

        # Parse if it's a string
        if isinstance(test_results, str):
            try:
                test_results = json.loads(test_results)
            except:
                test_results = {}

        if test_results and test_results.get('test_summary', {}).get('total_tests_run', 0) >
0:

            summary = test_results['test_summary']
            total = summary.get('total_tests_run', 0)
            passed = summary.get('tests_passed', 0)
            if total > 0:
                pass_rate = (passed / total) * 100
                state_updates[StateKeys.USER_LAST_TEST_PASS_RATE] = pass_rate

```

```

# Apply all updates atomically
for key, value in state_updates.items():
    tool_context.state[key] = value

logger.info(f"Tool: Progress updated - Attempt #{attempts}, "
           f"Lifetime: {lifetime_submissions}")

return {
    "status": "success",
    "session_attempts": attempts,
    "lifetime_submissions": lifetime_submissions,
    "timestamp": current_time,
    "improvement": {
        "style_score_change": score_improvement,
        "direction": "improved" if score_improvement > 0 else "declined"
    },
    "summary": f"Attempt #{attempts} recorded, {lifetime_submissions} total
submissions"
}

except Exception as e:
    error_msg = f"Progress update error: {str(e)}"
    logger.error(f"Tool: {error_msg}", exc_info=True)

return {
    "status": "error",
    "message": error_msg
}

```

生產模式：多層狀態管理

這個工具會示範 ADK 的三層狀態模型：

```
# Temporary (invocation-level) - cleared after this turn
state_updates[StateKeys.TEMP_PROCESSING_TIMESTAMP] = current_time

# Session-level - persists during this conversation
state_updates[StateKeys.GRADING_ATTEMPTS] = attempts

# User-level - persists across all sessions
state_updates[StateKeys.USER_TOTAL_SUBMISSIONS] = lifetime_submissions
```

為什麼要分成三種方案？

- **暫時：**偵錯資訊、時間戳記 - 本輪對話結束後不需要
- **工作階段：**目前的審查資料 - 審查結束前需要
- **使用者：**生命週期指標 - 跨工作階段的個人化設定需要這項指標

這項工具會比較目前與先前的評估結果，計算出改善幅度：

```
current_style_score = tool_context.state.get(StateKeys.STYLE_SCORE, 0)
last_style_score = tool_context.state.get(StateKeys.USER_LAST_STYLE_SCORE, 0)
score_improvement = current_style_score - last_style_score
```

這項功能可提供「自上次審查以來，你的風格進步了 15 分！」等意見回饋。

👉 尋找：

```
# MODULE_5_STEP_4_SAVE_GRADING_REPORT
```

👉 Replace with Tool 3 - Artifact Saver (production version):

```

async def save_grading_report(feedback_text: str, tool_context: ToolContext) -> Dict[str,
Any]:
    """
    Saves a detailed grading report as an artifact.

    Args:
        feedback_text: The feedback text to include in the report
        tool_context: ADK tool context for state management

    Returns:
        Dictionary containing save status and details
    """
    logger.info("Tool: Saving grading report...")

    try:
        # Gather all relevant data from state
        code = tool_context.state.get(StateKeys.CODE_TO_REVIEW, '')
        analysis = tool_context.state.get(StateKeys.CODE_ANALYSIS, {})
        style_score = tool_context.state.get(StateKeys.STYLE_SCORE, 0)
        style_issues = tool_context.state.get(StateKeys.STYLE_ISSUES, [])

        # Get test results
        test_results = tool_context.state.get(StateKeys.TEST_EXECUTION_SUMMARY, {})

        # Parse if it's a string
        if isinstance(test_results, str):
            try:
                test_results = json.loads(test_results)
            except:
                test_results = {}

        timestamp = datetime.now().isoformat()

        # Create comprehensive report dictionary
        report = {
            'timestamp': timestamp,
            'grading_attempt': tool_context.state.get(StateKeys.GRADING_ATTEMPTS, 1),
            'code': {
                'content': code,
                'line_count': len(code.splitlines()),
                'hash': hashlib.md5(code.encode()).hexdigest()
            },
            'analysis': analysis,
            'style': {
                'score': style_score,
                'issues': style_issues[:5] # First 5 issues
            },
            'tests': test_results,
            'feedback': feedback_text,
            'improvements': {
                'score_change': tool_context.state.get(StateKeys.SCORE_IMPROVEMENT, 0),
                'from_last_score': tool_context.state.get(StateKeys.USER_LAST_STYLE_SCORE, 0)
            }
        }
    
```

```

    }
}

# Convert report to JSON string
report_json = json.dumps(report, indent=2)
report_part = types.Part.from_text(text=report_json)

# Try to save as artifact if the service is available
if hasattr(tool_context, 'save_artifact'):
    try:
        # Generate filename with timestamp (replace colons for filesystem
compatibility)
        filename = f"grading_report_{timestamp.replace(':', '-')}.json"

        # Save the main report
        version = await tool_context.save_artifact(filename, report_part)

        # Also save a "latest" version for easy access
        await tool_context.save_artifact("latest_grading_report.json", report_part)

        logger.info(f"Tool: Report saved as {filename} (version {version})")

        # Store report in state as well for redundancy
        tool_context.state[StateKeys.USER_LAST_GRADING_REPORT] = report

    return {
        "status": "success",
        "artifact_saved": True,
        "filename": filename,
        "version": str(version),
        "size": len(report_json),
        "summary": f"Report saved as {filename}"
    }

    except Exception as artifact_error:
        logger.warning(f"Artifact service error: {artifact_error}, falling back to
state storage")
        # Continue to fallback below

# Fallback: Store in state if artifact service is not available or failed
tool_context.state[StateKeys.USER_LAST_GRADING_REPORT] = report
logger.info("Tool: Report saved to state (artifact service not available)")

return {
    "status": "success",
    "artifact_saved": False,
    "message": "Report saved to state only",
    "size": len(report_json),
    "summary": "Report saved to session state"
}

except Exception as e:

```

```

error_msg = f"Report save error: {str(e)}"
logger.error(f"Tool: {error_msg}", exc_info=True)

# Still try to save minimal data to state
try:
    tool_context.state[StateKeys.USER_LAST_GRADING_REPORT] = {
        'error': error_msg,
        'feedback': feedback_text,
        'timestamp': datetime.now().isoformat()
    }
except:
    pass

return {
    "status": "error",
    "message": error_msg,
    "artifact_saved": False,
    "summary": f"Failed to save report: {error_msg}"
}

```

製作模式：全面性報表

這份報表會匯總多個來源的資料：

```

report = {
    'code': {...},          # Original submission
    'analysis': {...},      # From code_analyzer
    'style': {...},         # From style_checker
    'tests': {...},         # From test_runner
    'feedback': {...},      # From this agent
    'improvements': {...}   # Calculated from history
}

```

這會建立審查程序的完整稽核記錄。

雙重儲存空間策略：

```

# Try artifact service first (persistent, downloadable)
if hasattr(tool_context, 'save_artifact'):
    await tool_context.save_artifact(filename, report_part)

# Fall back to state (always works)
tool_context.state[StateKeys.USER_LAST_GRADING_REPORT] = report

```

生產系統需要這項備援機制，因為如果構件儲存空間發生故障，資料就不會遺失。

建立 Synthesizer Agent

👉 開啟

```
code_review_assistant/sub_agents/review_pipeline/feedback_synthesizer.py
```

👉 尋找：

```
# MODULE_5_STEP_4_INSTRUCTION_PROVIDER
```

👉 請替換為生產指示供應商：

```
async def feedback_instruction_provider(context: ReadonlyContext) -> str:
    """Dynamic instruction provider that injects state variables."""
    template = """You are an expert code reviewer and mentor providing constructive,
educational feedback.
```

CONTEXT FROM PREVIOUS AGENTS:

- Structure analysis summary: {structure_analysis_summary}
- Style check summary: {style_check_summary}
- Test execution summary: {test_execution_summary}

YOUR TASK requires these steps IN ORDER:

1. Call search_past_feedback tool with developer_id="default_user"
2. Call update_grading_progress tool with no parameters
3. Carefully analyze the test results to understand what really happened
4. Generate comprehensive feedback following the structure below
5. Call save_grading_report tool with the feedback_text parameter
6. Return the feedback as your final output

CRITICAL - Understanding Test Results:

The test_execution_summary contains structured JSON. Parse it carefully:

- tests_passed = Code worked correctly
- tests_failed = Code produced wrong output
- tests_with_errors = Code crashed
- critical_issues = Fundamental problems with the code

If critical_issues array contains items, these are serious bugs that need fixing.
Do NOT count discovering bugs as test successes.

FEEDBACK STRUCTURE TO FOLLOW:

📄 Summary

Provide an honest assessment. Be encouraging but truthful about problems found.

✅ Strengths

List 2-3 things done well, referencing specific code elements.

📈 Code Quality Analysis

Structure & Organization

Comment on code organization, readability, and documentation.

Style Compliance

Report the actual style score and any specific issues.

Test Results

Report the actual test results accurately:

- If critical_issues exist, report them as bugs to fix
- Be clear: "X tests passed, Y critical issues were found"
- List each critical issue
- Don't hide or minimize problems

💡 Recommendations for Improvement

Based on the analysis, provide specific actionable fixes.
If critical issues exist, fixing them is top priority.

🚩 Next Steps

Prioritized action list based on severity of issues.

💬 Encouragement

End with encouragement while being honest about what needs fixing.

Remember: Complete ALL steps including calling save_grading_report."""

```
return await instructions_utils.inject_session_state(template, context)
```

👉 尋找：

```
# MODULE_5_STEP_4_SYNTHESIZER_AGENT
```

👉 替換為：

```
feedback_synthesizer_agent = Agent(  
    name="FeedbackSynthesizer",  
    model=config.critic_model,  
    description="Synthesizes all analysis into constructive, personalized feedback",  
    instruction=feedback_instruction_provider,  
    tools=[  
        FunctionTool(func=search_past_feedback),  
        FunctionTool(func=update_grading_progress),  
        FunctionTool(func=save_grading_report)  
    ],  
    output_key="final_feedback"  
)
```

製作模式：透過工具自動化調度管理提供結構化意見回饋

合成器會依序協調三項工具：

```
# 1. Search memory for patterns
search_past_feedback(developer_id="default_user")

# 2. Update progress metrics (no params - reads from state)
update_grading_progress()

# 3. Save comprehensive report
save_grading_report(feedback_text=generated_feedback)
```

為何這個順序很重要：

1. 記憶體搜尋 **第一次** - 先取得歷史記錄情境，再撰寫意見回饋
2. 進度更新 **中間** - 記錄指標，同時進行合成
3. 儲存報表**最後** - 擷取生成後的完整意見回饋

指令會明確告知 LLM 依序呼叫這些函式，確保行為一致。

合成的評論家模型

與測試執行器一樣，合成器會使用功能更強大的模型：

```
model=config.critic_model,
```

為什麼要使用昂貴的機型？合成器必須：

- 從測試執行器剖析 JSON (結構化資料)
- 區分錯誤報告和成功測試
- 將記憶體模式與目前結果整合
- 生成個人化的鼓勵意見回饋
- 誠實與鼓勵並重

這種細微的差異和推理能力足以證明費用合理。機械代理 (分析器、樣式檢查器) 可透過工作人員模型節省費用。

步驟 5：接上 Pipeline 的電線

現在，請將這四個代理程式全部連入循序管線，並建立根代理程式。

👉 開啟

```
code_review_assistant/agent.py
```

👉 在檔案頂端 (現有匯入項目後方) 新增必要匯入項目：

```

from google.adk.agents import Agent, SequentialAgent
from code_review_assistant.sub_agents.review_pipeline.code_analyzer import
code_analyzer_agent
from code_review_assistant.sub_agents.review_pipeline.style_checker import
style_checker_agent
from code_review_assistant.sub_agents.review_pipeline.test_runner import test_runner_agent
from code_review_assistant.sub_agents.review_pipeline.feedback_synthesizer import
feedback_synthesizer_agent

```

現在檔案應如下所示：

```

"""
Main agent orchestration for the Code Review Assistant.
"""

from google.adk.agents import Agent, SequentialAgent
from .config import config
from code_review_assistant.sub_agents.review_pipeline.code_analyzer import
code_analyzer_agent
from code_review_assistant.sub_agents.review_pipeline.style_checker import
style_checker_agent
from code_review_assistant.sub_agents.review_pipeline.test_runner import test_runner_agent
from code_review_assistant.sub_agents.review_pipeline.feedback_synthesizer import
feedback_synthesizer_agent

# MODULE_5_STEP_5_CREATE_PIPELINE

# MODULE_6_STEP_5_CREATE_FIX_LOOP

# MODULE_6_STEP_5_UPDATE_ROOT_AGENT

```

👉 尋找：

```
# MODULE_5_STEP_5_CREATE_PIPELINE
```

👉 將該單行程式碼替換為：

```

# Create sequential pipeline
code_review_pipeline = SequentialAgent(
    name="CodeReviewPipeline",
    description="Complete code review pipeline with analysis, testing, and feedback",
    sub_agents=[
        code_analyzer_agent,
        style_checker_agent,
        test_runner_agent,
        feedback_synthesizer_agent
    ]
)

# Root agent - coordinates the review pipeline
root_agent = Agent(
    name="CodeReviewAssistant",
    model=config.worker_model,
    description="An intelligent code review assistant that analyzes Python code and provides educational feedback",
    instruction="""You are a specialized Python code review assistant focused on helping developers improve their code quality.

When a user provides Python code for review:
1. Immediately delegate to CodeReviewPipeline and pass the code EXACTLY as it was provided by the user.
2. The pipeline will handle all analysis and feedback
3. Return ONLY the final feedback from the pipeline - do not add any commentary

When a user asks what you can do or asks general questions:
- Explain your capabilities for code review
- Do NOT trigger the pipeline for non-code messages

The pipeline handles everything for code review - just pass through its final output.""",
    sub_agents=[code_review_pipeline],
    output_key="assistant_response"
)

```

循序管道依附元件

管道順序至關重要：

```
agents=[
    code_analyzer_agent,      # Creates CODE_TO_REVIEW in state
    style_checker_agent,      # Reads CODE_TO_REVIEW
    test_runner_agent,        # Reads CODE_TO_REVIEW
    feedback_synthesizer_agent # Reads all three summaries
]
```

各個代理程式的需求條件：

- **Analyzer**：只需要使用者輸入內容 → 最先執行
- **樣式/測試**：需要分析器提供的 `CODE_TO_REVIEW`
- **合成器**：需要所有三種 `output_key` 摘要

重新排序會破壞依附元件。如果先執行樣式檢查器：

```
code = tool_context.state.get(StateKeys.CODE_TO_REVIEW) # Returns None!
```

第 5 集與最終製作

完成單元 5 後，您的 `agent.py` 應會包含：

```
# Module 5 - Review pipeline only
root_agent = Agent(
    sub_agents=[code_review_pipeline] # Single pipeline
)
```

在第 6 個單元中，您將新增修正管道：

```
# Module 6 - Both pipelines
root_agent = Agent(
    sub_agents=[code_review_pipeline, code_fix_pipeline] # Two pipelines
)
```

指令也會展開，提供修正方式並處理使用者回覆。目前請先著重於讓審查管道正常運作。

步驟 6：測試完整管道

現在來看看這四個代理如何協同運作。

👉 啟動系統：

```
adk web code_review_assistant
```

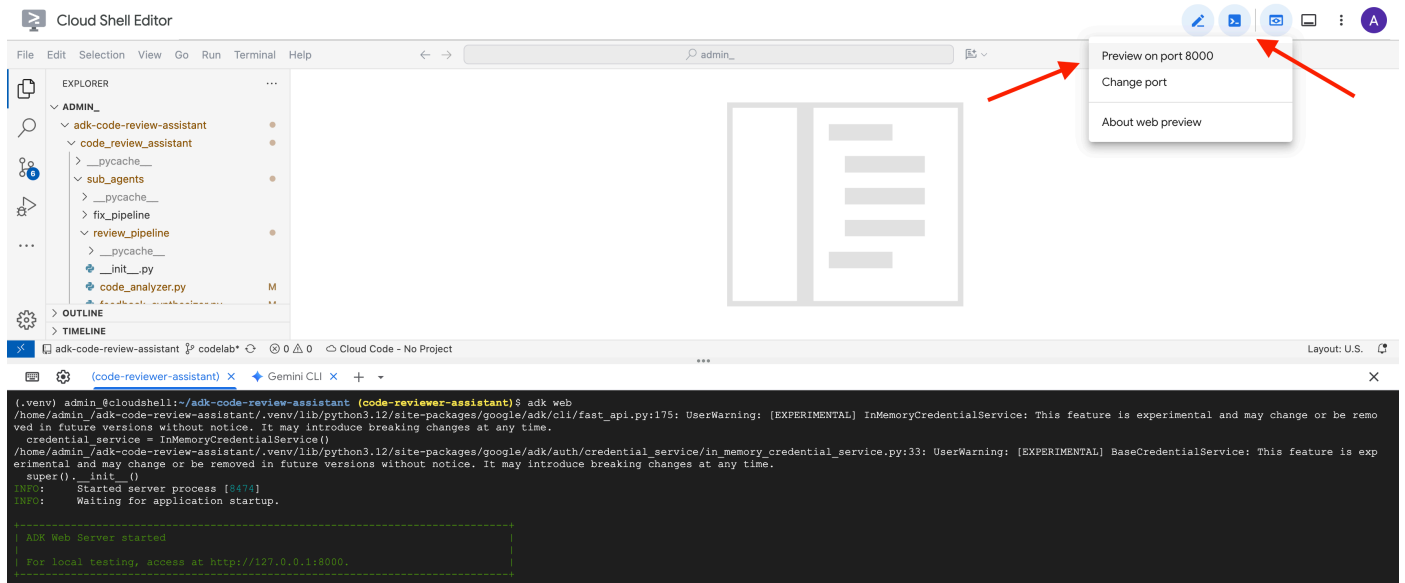
執行 `adk web` 指令後，終端機應會顯示 ADK 網頁伺服器已啟動的輸出內容，如下所示：

```
+-----+
| ADK Web Server started                                |
|                                                       |
| For local testing, access at http://localhost:8000.   |
+-----+

INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

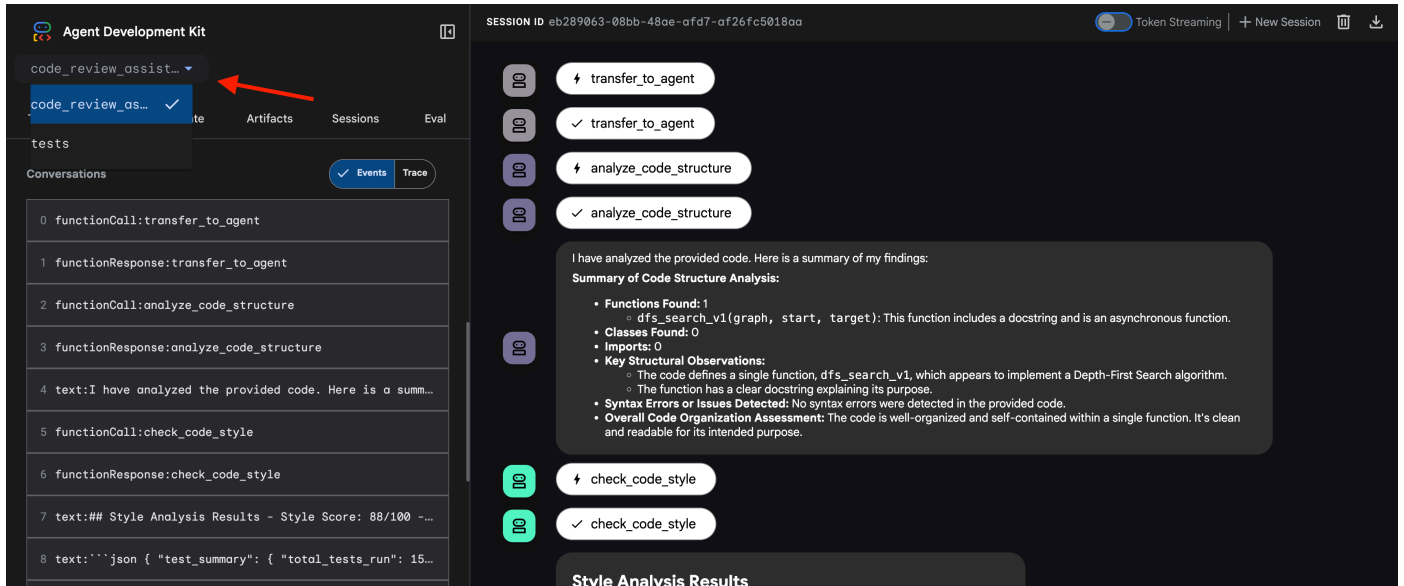
👉 接著，如要從瀏覽器存取 ADK 開發人員使用者介面，請按照下列步驟操作：

在 Cloud Shell 工具列 (通常位於右上角) 中，按一下「網頁預覽」圖示 (通常看起來像眼睛或帶有箭頭的方塊)，然後選取「變更通訊埠」。在彈出式視窗中，將通訊埠設為 8000，然後按一下「變更並預覽」。Cloud Shell 隨即會開啟新的瀏覽器分頁或視窗，顯示 ADK 開發人員使用者介面。



👉 代理程式現在正在執行。瀏覽器中的 **ADK 開發人員使用者介面** 是與代理程式的直接連線。

- 選取目標：在 UI 頂端的下拉式選單中，選擇 `code_review_assistant` 代理程式。



👉 測試提示：

```
Please analyze the following:
def dfs_search_v1(graph, start, target):
    """Find if target is reachable from start."""
    visited = set()
    stack = start

    while stack:
        current = stack.pop()

        if current == target:
            return True

        if current not in visited:
            visited.add(current)

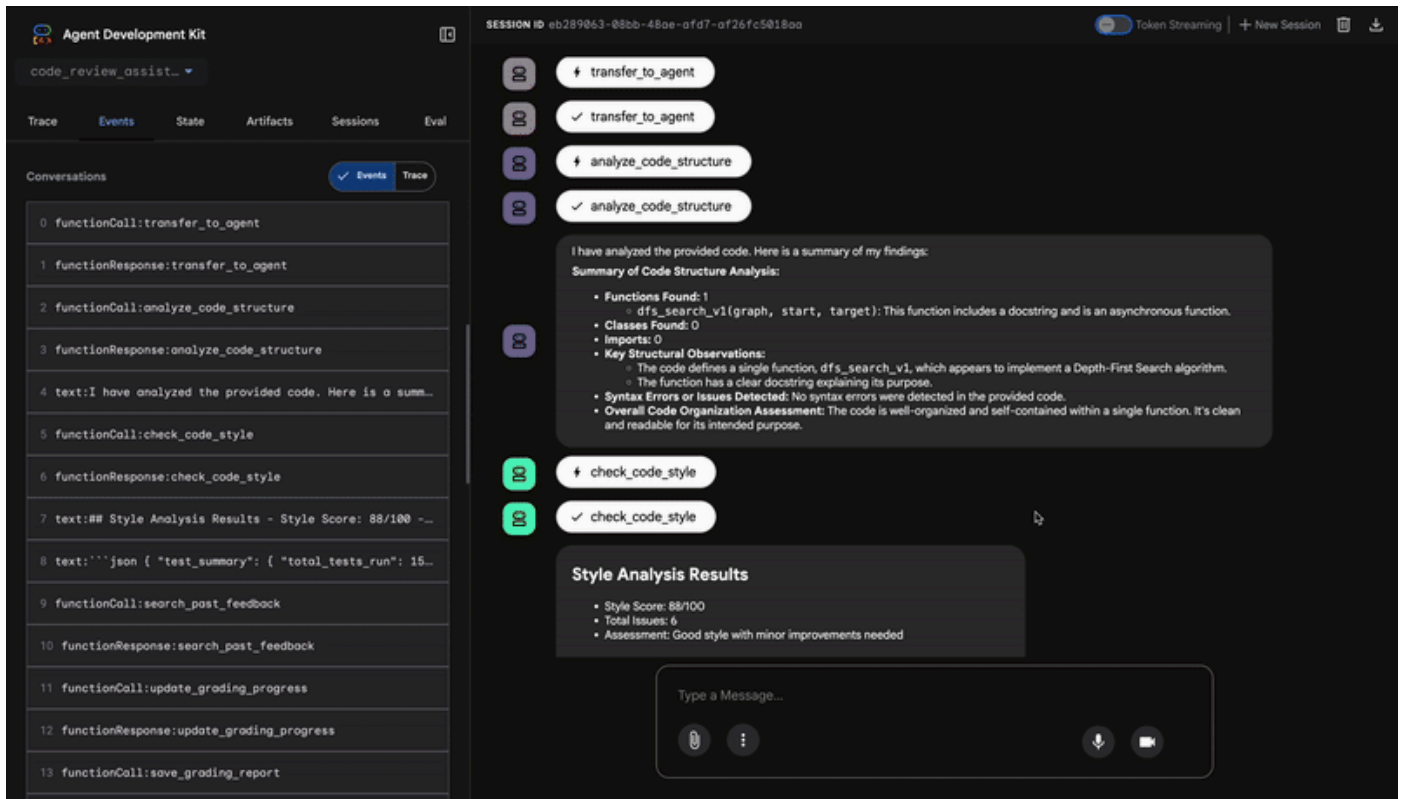
            for neighbor in graph[current]:
                if neighbor not in visited:
                    stack.append(neighbor)

    return False
```

👉 查看程式碼審查管道的實際運作情形：

提交有錯誤的 `dfs_search_v1` 函式時，您不只會收到一個答案，您正在見證多代理管道的運作。您看到的串流輸出內容，是四個專業代理程式依序執行的結果，每個代理程式都會以前一個代理程式的結果為基礎。

以下是各個代理程式對最終全面審查的貢獻，可將原始資料轉化為可做為行動依據的情報。



1. 程式碼分析工具的結構報表

首先，`CodeAnalyzer` 代理程式會收到原始程式碼。不會猜測程式碼的作用，而是使用 `analyze_code_structure` 工具執行確定性的抽象語法樹 (AST) 剖析。

輸出內容是程式碼結構的純粹事實資料：

```
The analysis of the provided code reveals the following:
```

```
Summary:
```

- Functions Found: 1
- Classes Found: 0

```
Key Structural Observations:
```

- A single function, `dfs_search_v1`, is defined.
- It includes a docstring: "Find if target is reachable from start."
- No syntax errors were detected.

```
Overall Code Organization Assessment:
```

- The code snippet is a well-defined, self-contained function.

★ 價值：這個初始步驟可為其他代理程式提供乾淨可靠的基礎。確認程式碼是否為有效的 Python 程式碼，並找出需要審查的確切元件。

2. Style Checker 的 PEP 8 稽核

接著，`StyleChecker` 代理程式會接手。這個工具會從共用狀態讀取程式碼，並使用 `check_code_style` 工具 (該工具會運用 `pycodestyle` Linter)。

輸出結果為可量化的品質分數和具體違規事項：

Style Analysis Results

- Style Score: 88/100
- Total Issues: 6
- Assessment: Good style with minor improvements needed

Top Style Issues

- Line 5, W293: blank line contains whitespace
- Line 19, W292: no newline at end of file

★ **價值：**這個代理程式會根據既定的社群標準 (PEP 8) 提供客觀且不可協商的回饋。加權評分系統會立即向使用者說明問題的嚴重程度。

3. 測試執行工具發現的重大錯誤

這時系統會進行更深入的分析，`TestRunner` 代理程式會生成並執行一系列全面測試，驗證程式碼的行為。

輸出內容是結構化 **JSON** 物件，其中包含不利的判決：

```
{
  "critical_issues": [
    {
      "type": "Critical Bug",
      "description": "The function's initialization `stack = start` is incorrect... When a common input like a string... is provided... the function crashes with an AttributeError.",
      "severity": "Critical"
    }
  ],
  "verdict": {
    "status": "BROKEN",
    "confidence": "high",
    "recommendation": "The function is fundamentally broken... the stack initialization line `stack = start` must be changed to `stack = [start]`."
  }
}
```

★ **價值：**這是最重要的洞察資料。代理不只是猜測，而是執行程式碼，證明程式碼有問題。這項工具發現了細微但至關重要的執行階段錯誤，這類錯誤很容易被人工審查員忽略，並準確指出錯誤原因和必要修正。

4. 意見回饋綜合分析工具的最終報告

最後，`FeedbackSynthesizer` 代理程式會擔任指揮的角色，這項工具會從前三位服務專員的對話中擷取結構化資料，然後製作一份簡單易懂的報告，內容兼具分析性與鼓勵性。

最終輸出內容就是您看到的最終潤飾版評論：

Summary

Great effort on implementing the Depth-First Search algorithm! ... However, a critical bug in the initialization of the stack prevents the function from working correctly...

Strengths

- Good Algorithm Structure
- Correct Use of `visited` Set

Code Quality Analysis

...

Style Compliance

The style analysis returned a good score of 88/100.

...

Test Results

The automated testing revealed a critical issue... The line `stack = start` directly assigns the input... which results in an `AttributeError`.

Recommendations for Improvement

****Fix the Critical Stack Initialization Bug:****

- Incorrect Code: `stack = start`
- Correct Code: `stack = [start]`

Encouragement

You are very close to a perfect implementation! The core logic of your DFS algorithm is sound, which is the hardest part.

★ **價值：**這個代理程式會將技術資料轉換為實用的教育體驗。這封郵件會優先處理最重要的問題 (錯誤)，清楚說明問題，提供確切的解決方案，並以鼓勵的語氣撰寫。並將先前各階段的發現整合為連貫且有價值的整體。

這個多級式程序展現了代理管道的強大功能。您會收到分層分析結果，而不是單一的整體回覆，因為每個代理程式都會執行可驗證的專業工作。這項審查不僅深入，而且具決定性、可靠且深入教育意義。

👉  測試完成後，請返回 Cloud Shell 編輯器終端機，然後按下 `Ctrl+C` 停止 ADK 開發人員使用者介面。

建構項目

您現在擁有完整的程式碼審查管道，可執行下列作業：

- ✅ **剖析程式碼結構** - 使用輔助函式進行確定性 AST 分析
- ✅ **檢查樣式** - 使用命名慣例進行加權評分
- ✅ **執行測試** - 產生全面性測試，並以 JSON 格式輸出結構化內容
- ✅ **綜合意見回饋** - 整合狀態、記憶體和構件
- ✅ **追蹤進度** - 跨呼叫/工作階段/使用者的多層狀態
- ✅ **隨著時間學習** - 記憶體服務，可跨工作階段找出模式
- ✅ **提供構件** - 可下載的 JSON 報表，內含完整的稽核追蹤記錄

已掌握的關鍵概念

循序管道：

- 四個代理會依嚴格順序執行
- 每個狀態都會豐富下一個狀態
- 依附元件決定執行順序

生產模式：

- 輔助函式分離 (在執行緒集區中同步)
- 優雅降級 (備援策略)
- 多層級狀態管理 (暫時/工作階段/使用者)
- 動態指令提供者 (會根據狀態調整內容)
- 雙重儲存空間 (構件 + 狀態備援)

以通訊形式呈現狀態：

- 常數可避免代理程式發生錯別字
- `output_key` 將代理摘要寫入狀態
- 後續代理會透過 `StateKeys` 讀取
- 狀態會以線性方式流經管道

記憶體與狀態：

- 狀態：目前的工作階段資料
- 記憶：跨工作階段的模式
- 用途不同，生命週期也不同

工具自動化調度管理：

- 單一工具代理程式 (分析器、樣式檢查器)
- 內建執行器 (`test_runner`)
- 協調多種工具 (合成器)

模型選取策略：

- 工作人員模型：機械工作 (剖析、檢查、路徑)
- 評論家模型：推理工作 (測試、綜合)
- 透過適當的選取項目進行成本最佳化

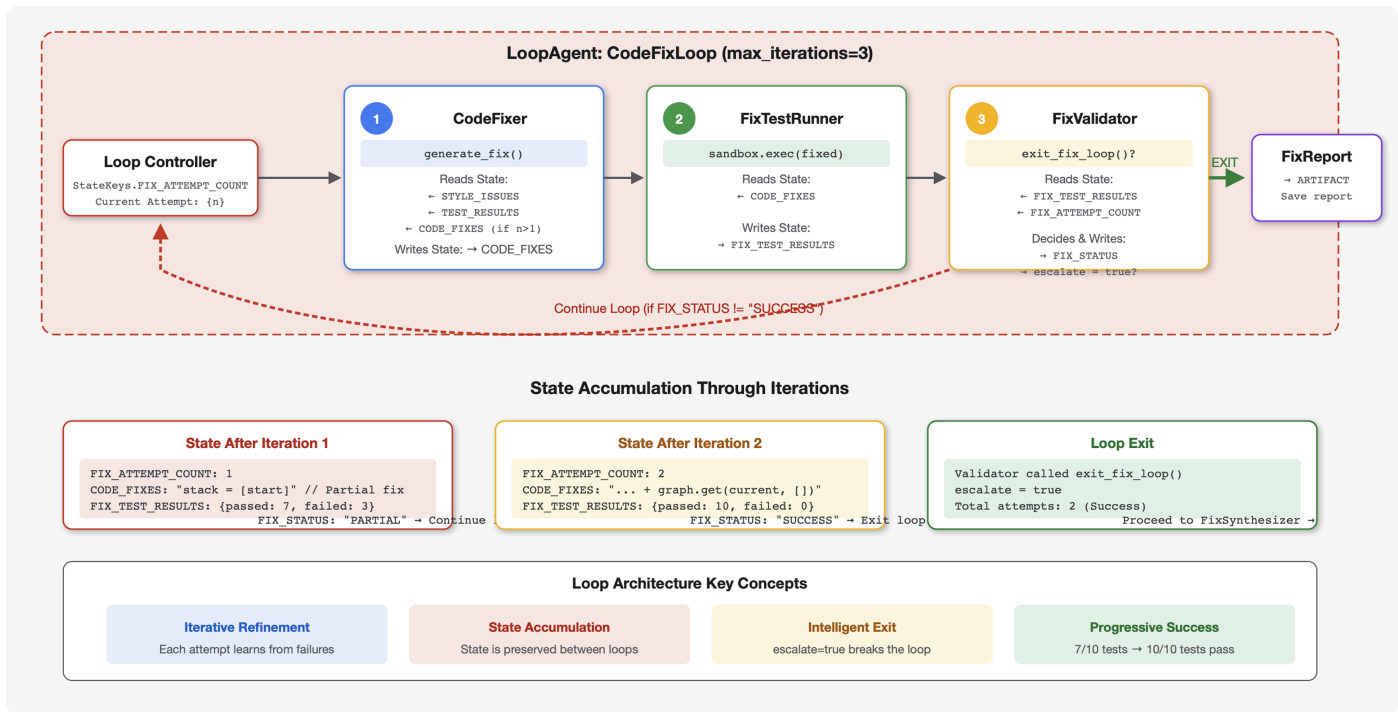
後續步驟

在單元 6 中，您將建構**修正管道**：

- **LoopAgent 架構**，可反覆修正
- 透過提報退出條件
- **狀態累積** (跨疊代)
- **驗證和重試邏輯**
- **整合審查管道**，提供修正建議

您將瞭解如何將相同的狀態模式擴展至複雜的疊代工作流程，讓代理程式多次嘗試直到成功，以及如何在單一應用程式中協調多個管道。

6. 新增修正管道：迴圈架構



簡介

在第 5 模組中，您建構了循序審查管道，可分析程式碼並提供意見回饋。但找出問題只是解決方案的一半，開發人員還需要協助修正問題。

本單元會建構自動修正管道，可執行下列作業：

1. 根據審查結果生成修正內容
2. 透過執行全面測試來驗證修正結果
3. 如果修正方式無效，系統會自動重試 (最多 3 次)
4. 報表結果 (比較前後成效)

重要概念：使用 **LoopAgent** 自動重試。與只執行一次的循序代理不同，**LoopAgent** 會重複執行子代理，直到符合結束條件或達到疊代次數上限為止。工具會設定 `tool_context.actions.escalate = True`，表示成功。

您將建構的內容預覽：提交有錯誤的程式碼 → 檢查找出問題 → 修正迴圈生成修正內容 → 測試驗證 → 視需要重試 → 最終的完整報表。

核心概念：LoopAgent 與 Sequential

循序管道 (模組 5)：

```
SequentialAgent(agents=[A, B, C])
# Executes: A → B → C → Done
```

- 單向流程
- 每個代理只會執行一次
- 沒有重試邏輯

迴圈管道 (單元 6)：

```
LoopAgent(agents=[A, B, C], max_iterations=3)
# Executes: A → B → C → (check exit) → A → B → C → (check exit) → ...
```

- 循環流程
- 代理可以多次執行
- 退出條件：
 - 工具會設定 `tool_context.actions.escalate = True` (成功)
 - 已達 `max_iterations` (安全限制)
 - 發生未處理的例外狀況 (錯誤)

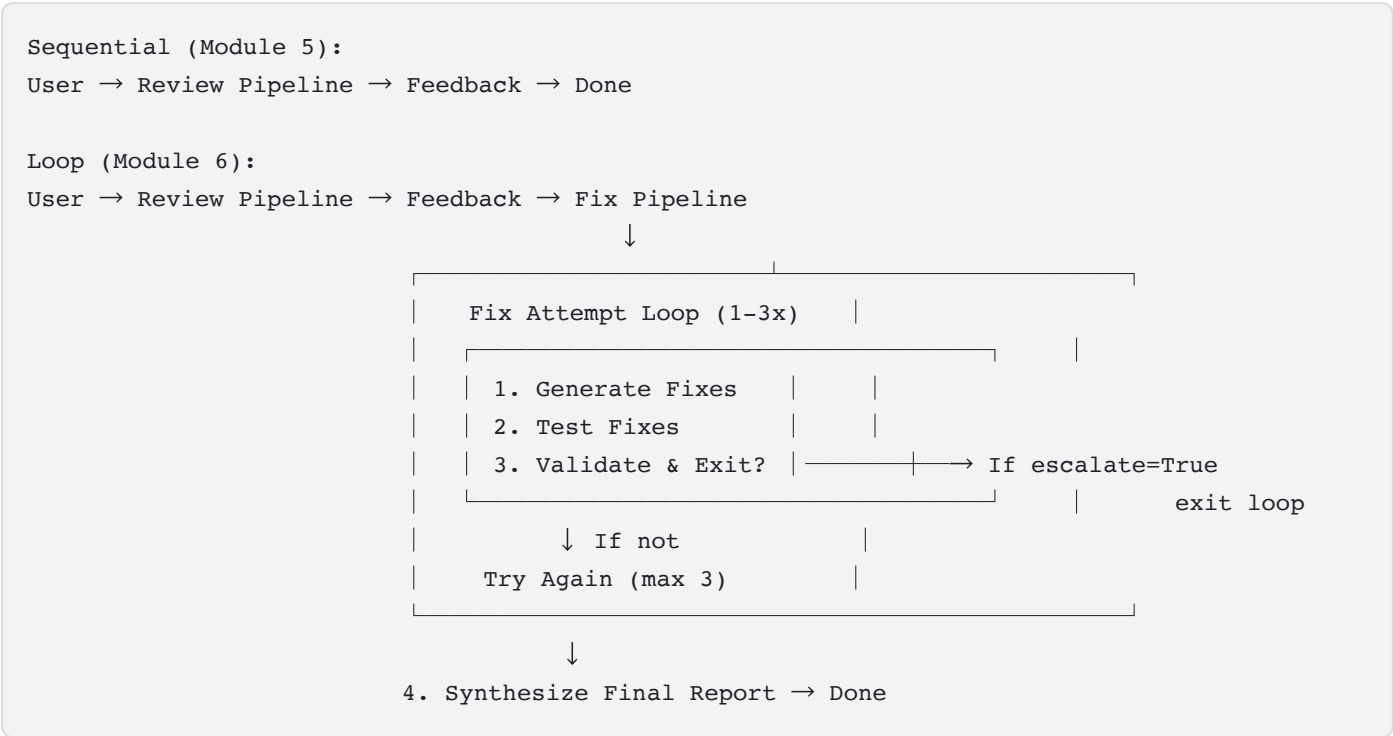
為什麼要使用迴圈修正式碼：

通常需要多次嘗試才能修正式碼：

- **第一次嘗試**：修正明顯的錯誤 (變數類型錯誤)
- **第二次嘗試**：修正測試發現的次要問題 (極端情況)
- **第三次嘗試**：微調並驗證所有測試是否通過

如果沒有迴圈，您就需要在代理程式指令中加入複雜的條件邏輯。使用 `LoopAgent` 時，系統會自動重試。

架構比較：



步驟 1：新增 Code Fixer 代理程式

程式碼修正工具會根據審查結果生成修正後的 Python 程式碼。

👉 開啟

```
code_review_assistant/sub_agents/fix_pipeline/code_fixer.py
```

👉 尋找：

```
# MODULE_6_STEP_1_CODE_FIXER_INSTRUCTION_PROVIDER
```

👉 將該單行程式碼替換為：

```
async def code_fixer_instruction_provider(context: ReadonlyContext) -> str:
    """Dynamic instruction provider that injects state variables."""
    template = """You are an expert code fixing specialist.

Original Code:
{code_to_review}

Analysis Results:
- Style Score: {style_score}/100
- Style Issues: {style_issues}
- Test Results: {test_execution_summary}

Based on the test results, identify and fix ALL issues including:
- Interface bugs (e.g., if start parameter expects wrong type)
- Logic errors (e.g., KeyError when accessing graph nodes)
- Style violations
- Missing documentation

YOUR TASK:
Generate the complete fixed Python code that addresses all identified issues.

CRITICAL INSTRUCTIONS:
- Output ONLY the corrected Python code
- Do NOT include markdown code blocks (```python)
- Do NOT include any explanations or commentary
- The output should be valid, executable Python code and nothing else

Common fixes to apply based on test results:
- If tests show AttributeError with 'pop', fix: stack = [start] instead of stack = start
- If tests show KeyError accessing graph, fix: use graph.get(current, [])
- Add docstrings if missing
- Fix any style violations identified

Output the complete fixed code now:"""

    return await instructions_utils.inject_session_state(template, context)
```

👉 尋找：

```
# MODULE_6_STEP_1_CODE_FIXER_AGENT
```

👉 將該單行程式碼替換為：

```
code_fixer_agent = Agent(
    name="CodeFixer",
    model=config.worker_model,
    description="Generates comprehensive fixes for all identified code issues",
    instruction=code_fixer_instruction_provider,
    code_executor=BuiltInCodeExecutor(),
    output_key="code_fixes"
)
```

為什麼只能輸出原始程式碼？

指令明確指出「請勿加入 Markdown 程式碼區塊」：錯誤的輸出內容 (會導致下游代理程式發生錯誤)：

```
def fixed_function():
    pass
```

輸出結果良好 (原始 Python)：

```
def fixed_function():
    pass
```

這是因為 `fix_test_runner_agent` 必須直接執行這段程式碼。Markdown 格式會導致語法錯誤。 `output_key="code_fixes"` 會在狀態中儲存原始 Python。

再次瞭解 Context Provider 模式

與第 5 模組中的合成器一樣，修正器會使用動態指令：

```
instruction=code_fixer_instruction_provider
```

函式會在每次叫用時讀取目前狀態：

- `{code_to_review}` - original buggy code
- `{style_issues}` - 修正內容
- `{test_execution_summary}` - 失敗原因

如果迴圈重試，指令會看到先前嘗試的更新狀態。

步驟 2：新增 Fix Test Runner Agent

修正測試執行器會對修正後的程式碼執行全面測試，藉此驗證修正內容。

👉 開啟

```
code_review_assistant/sub_agents/fix_pipeline/fix_test_runner.py
```

👉 尋找：

```
# MODULE_6_STEP_2_FIX_TEST_RUNNER_INSTRUCTION_PROVIDER
```

👉 將該單行程式碼替換為：

```
async def fix_test_runner_instruction_provider(context: ReadonlyContext) -> str:
    """Dynamic instruction provider that uses the clean code from the previous step."""
    template = """You are responsible for validating the fixed code by running tests.

THE FIXED CODE TO TEST:
{code_fixes}

ORIGINAL TEST RESULTS: {test_execution_summary}

YOUR TASK:
1. Understand the fixes that were applied
2. Generate the same comprehensive tests (15-20 test cases)
3. Execute the tests on the FIXED code using your code executor
4. Compare results with original test results
5. Output a detailed JSON analysis

TESTING METHODOLOGY:
- Run the same tests that revealed issues in the original code
- Verify that previously failing tests now pass
- Ensure no regressions were introduced
- Document the improvement

Execute your tests and output ONLY valid JSON with this structure:
- "passed": number of tests that passed
- "failed": number of tests that failed
- "total": total number of tests
- "pass_rate": percentage as a number
- "comparison": object with "original_pass_rate", "new_pass_rate", "improvement"
- "newly_passing_tests": array of test names that now pass
- "still_failing_tests": array of test names still failing

Do NOT output the test code itself, only the JSON analysis."""

    return await instructions_utils.inject_session_state(template, context)
```

👉 尋找：

```
# MODULE_6_STEP_2_FIX_TEST_RUNNER_AGENT
```

👉 將該單行程式碼替換為：


```
fix_test_runner_agent = Agent(
    name="FixTestRunner",
    model=config.critic_model,
    description="Runs comprehensive tests on fixed code to verify all issues are resolved",
    instruction=fix_test_runner_instruction_provider,
    code_executor=BuiltInCodeExecutor(),
    output_key="fix_test_execution_summary"
)
```

為什麼要使用 JSON 輸出格式？

這項指令需要結構化 JSON 輸出內容，並包含「passed」、「failed」、「total」、「pass_rate」和「comparison」等特定欄位。

為什麼要使用 JSON 而不是自然語言？

- 驗證器代理程式需要剖析數值
- 所有疊代都採用一致的結構
- 輕鬆比較改善成效

`output_key="fix_test_execution_summary"` 會將這個 JSON 儲存在狀態中。

測試用的評論家模型

請注意，這個代理會使用 `config.critic_model`：

```
model=config.critic_model,
```

這通常是功能更強大的模型 (例如 `gemini-2.5-pro`)，因為：

- 生成 15 到 20 個詳盡的測試案例需要高深的專業知識
- 必須瞭解特殊情況和潛在回歸
- 需要剖析原始測試結果並準確比較

修正工具使用 `worker_model`，因為程式碼生成更具機械性。測試需要批判性思考。

步驟 3：新增修正驗證器代理程式

驗證器會檢查修正是否成功，並決定是否要結束迴圈。

瞭解工具

首先，請新增驗證器所需的三項工具。

👉 開啟

```
code_review_assistant/tools.py
```

👉 尋找：

```
# MODULE_6_STEP_3_VALIDATE_FIXED_STYLE
```

👉 取代工具 1 - 樣式驗證工具：

```

async def validate_fixed_style(tool_context: ToolContext) -> Dict[str, Any]:
    """
    Validates style compliance of the fixed code.

    Args:
        tool_context: ADK tool context containing fixed code in state

    Returns:
        Dictionary with style validation results
    """
    logger.info("Tool: Validating style of fixed code...")

    try:
        # Get the fixed code from state
        code_fixes = tool_context.state.get(StateKeys.CODE_FIXES, '')

        # Try to extract from markdown if present
        if '```python' in code_fixes:
            start = code_fixes.rfind('```python') + 9
            end = code_fixes.rfind('```')
            if start < end:
                code_fixes = code_fixes[start:end].strip()

        if not code_fixes:
            return {
                "status": "error",
                "message": "No fixed code found in state"
            }

        # Store the extracted fixed code
        tool_context.state[StateKeys.CODE_FIXES] = code_fixes

        # Run style check on fixed code
        loop = asyncio.get_event_loop()
        with ThreadPoolExecutor() as executor:
            style_result = await loop.run_in_executor(
                executor, _perform_style_check, code_fixes
            )

        # Compare with original
        original_score = tool_context.state.get(StateKeys.STYLE_SCORE, 0)
        improvement = style_result['score'] - original_score

        # Store results
        tool_context.state[StateKeys.FIXED_STYLE_SCORE] = style_result['score']
        tool_context.state[StateKeys.FIXED_STYLE_ISSUES] = style_result['issues']

        logger.info(f"Tool: Fixed code style score: {style_result['score']}/100 "
                    f"(improvement: +{improvement})")

    return {
        "status": "success",
    }

```

```
        "fixed_style_score": style_result['score'],
        "original_style_score": original_score,
        "improvement": improvement,
        "remaining_issues": style_result['issues'],
        "perfect_style": style_result['score'] == 100
    }

except Exception as e:
    logger.error(f"Tool: Style validation failed: {e}", exc_info=True)
    return {
        "status": "error",
        "message": str(e)
    }
```

👉 尋找：

```
# MODULE_6_STEP_3_COMPILE_FIX_REPORT
```

👉 改用工具 2 - 報表編譯器：

```

async def compile_fix_report(tool_context: ToolContext) -> Dict[str, Any]:
    """
    Compiles comprehensive report of the fix process.

    Args:
        tool_context: ADK tool context with all fix pipeline data

    Returns:
        Comprehensive fix report
    """
    logger.info("Tool: Compiling comprehensive fix report...")

    try:
        # Gather all data
        original_code = tool_context.state.get(StateKeys.CODE_TO_REVIEW, '')
        code_fixes = tool_context.state.get(StateKeys.CODE_FIXES, '')

        # Test results
        original_tests = tool_context.state.get(StateKeys.TEST_EXECUTION_SUMMARY, {})
        fixed_tests = tool_context.state.get(StateKeys.FIX_TEST_EXECUTION_SUMMARY, {})

        # Parse if strings
        if isinstance(original_tests, str):
            try:
                original_tests = json.loads(original_tests)
            except:
                original_tests = {}

        if isinstance(fixed_tests, str):
            try:
                fixed_tests = json.loads(fixed_tests)
            except:
                fixed_tests = {}

        # Extract pass rates
        original_pass_rate = 0
        if original_tests:
            if 'pass_rate' in original_tests:
                original_pass_rate = original_tests['pass_rate']
            elif 'test_summary' in original_tests:
                # Handle test_runner_agent's JSON structure
                summary = original_tests['test_summary']
                total = summary.get('total_tests_run', 0)
                passed = summary.get('tests_passed', 0)
                if total > 0:
                    original_pass_rate = (passed / total) * 100
            elif 'passed' in original_tests and 'total' in original_tests:
                if original_tests['total'] > 0:
                    original_pass_rate = (original_tests['passed'] / original_tests['total'])

        * 100

        fixed_pass_rate = 0

```

```

all_tests_pass = False
if fixed_tests:
    if 'pass_rate' in fixed_tests:
        fixed_pass_rate = fixed_tests['pass_rate']
        all_tests_pass = fixed_tests.get('failed', 1) == 0
    elif 'passed' in fixed_tests and 'total' in fixed_tests:
        if fixed_tests['total'] > 0:
            fixed_pass_rate = (fixed_tests['passed'] / fixed_tests['total']) * 100
        all_tests_pass = fixed_tests.get('failed', 0) == 0

# Style scores
original_style = tool_context.state.get(StateKeys.STYLE_SCORE, 0)
fixed_style = tool_context.state.get(StateKeys.FIXED_STYLE_SCORE, 0)

# Calculate improvements
test_improvement = {
    'original_pass_rate': original_pass_rate,
    'fixed_pass_rate': fixed_pass_rate,
    'improvement': fixed_pass_rate - original_pass_rate,
    'all_tests_pass': all_tests_pass
}

style_improvement = {
    'original_score': original_style,
    'fixed_score': fixed_style,
    'improvement': fixed_style - original_style,
    'perfect_style': fixed_style == 100
}

# Determine overall status
if all_tests_pass and style_improvement['perfect_style']:
    fix_status = 'SUCCESSFUL'
    status_emoji = '✅'
elif test_improvement['improvement'] > 0 or style_improvement['improvement'] > 0:
    fix_status = 'PARTIAL'
    status_emoji = '⚠️'
else:
    fix_status = 'FAILED'
    status_emoji = '❌'

# Build comprehensive report
report = {
    'status': fix_status,
    'status_emoji': status_emoji,
    'timestamp': datetime.now().isoformat(),
    'original_code': original_code,
    'code_fixes': code_fixes,
    'improvements': {
        'tests': test_improvement,
        'style': style_improvement
    },
    'summary': f"{status_emoji} Fix Status: {fix_status}\n"

```

```

        f"Tests: {original_pass_rate:.1f}% → {fixed_pass_rate:.1f}%\n"
        f"Style: {original_style}/100 → {fixed_style}/100"
    }

    # Store report in state
    tool_context.state[StateKeys.FIX_REPORT] = report
    tool_context.state[StateKeys.FIX_STATUS] = fix_status

    logger.info(f"Tool: Fix report compiled - Status: {fix_status}")
    logger.info(f"Tool: Test improvement: {original_pass_rate:.1f}% → {fixed_pass_rate:.1f}%")
    logger.info(f"Tool: Style improvement: {original_style} → {fixed_style}")

    return {
        "status": "success",
        "fix_status": fix_status,
        "report": report
    }

except Exception as e:
    logger.error(f"Tool: Failed to compile fix report: {e}", exc_info=True)
    return {
        "status": "error",
        "message": str(e)
    }

```

👉 尋找：

```
# MODULE_6_STEP_3_EXIT_FIX_LOOP
```

👉 以工具 3 取代 - 迴圈結束信號：

```
def exit_fix_loop(tool_context: ToolContext) -> Dict[str, Any]:
    """
    Signal that fixing is complete and should exit the loop.

    Args:
        tool_context: ADK tool context

    Returns:
        Confirmation message
    """
    logger.info("Tool: Setting escalate flag to exit fix loop")

    # This is the critical line that exits the LoopAgent
    tool_context.actions.escalate = True

    return {
        "status": "success",
        "message": "Fix complete, exiting loop"
    }
```

提報機制

`exit_fix_loop` 工具有一行重要程式碼：

```
tool_context.actions.escalate = True
```

這會發出信號，要求 `LoopAgent` 停止疊代：

- 不升級：迴圈會繼續下一次疊代
- 使用 **escalate**：目前疊代完成後，迴圈會立即結束

為什麼是 `escalate`，而不是傳回特殊值？

- 管道中的任何工具都可以設定 (不只是最後一個)
- 所有代理程式類型都適用
- 清楚的語意含義：「escalate out of this loop」
- 不會干擾工具傳回的資料

驗證器會根據修正品質決定呼叫這項工具的時機。

建立驗證器代理程式

👉 開啟

```
code_review_assistant/sub_agents/fix_pipeline/fix_validator.py
```

👉 尋找：

```
# MODULE_6_STEP_3_FIX_VALIDATOR_INSTRUCTION_PROVIDER
```


👉 將該單行程式碼替換為：

```
async def fix_validator_instruction_provider(context: ReadonlyContext) -> str:
    """Dynamic instruction provider that injects state variables."""
    template = """You are the final validation specialist for code fixes.

You have access to:
- Original issues from initial review
- Applied fixes: {code_fixes}
- Test results after fix: {fix_test_execution_summary}
- All state data from the fix process

Your responsibilities:
1. Use validate_fixed_style tool to check style compliance of fixed code
    - Pass no arguments, it will retrieve fixed code from state
2. Use compile_fix_report tool to generate comprehensive report
    - Pass no arguments, it will gather all data from state
3. Based on the report, determine overall fix status:
    - ✅ SUCCESSFUL: All tests pass, style score 100
    - ⚠️ PARTIAL: Improvements made but issues remain
    - ❌ FAILED: Fix didn't work or made things worse

4. CRITICAL: If status is SUCCESSFUL, call the exit_fix_loop tool to stop iterations
    - This prevents unnecessary additional fix attempts
    - If not successful, the loop will continue for another attempt

5. Provide clear summary of:
    - What was fixed
    - What improvements were achieved
    - Any remaining issues requiring manual attention

Be precise and quantitative in your assessment.
"""

    return await instructions_utils.inject_session_state(template, context)
```

👉 尋找：

```
# MODULE_6_STEP_3_FIX_VALIDATOR_AGENT
```

👉 將該單行程式碼替換為：

```
fix_validator_agent = Agent(
    name="FixValidator",
    model=config.worker_model,
    description="Validates fixes and generates final fix report",
    instruction=fix_validator_instruction_provider,
    tools=[
        FunctionTool(func=validate_fixed_style),
        FunctionTool(func=compile_fix_report),
        FunctionTool(func=exit_fix_loop)
    ],
    output_key="final_fix_report"
)
```

三種工具，一個決定

驗證器會依序自動調度三項工具：

1. `validate_fixed_style()`

- 檢查：修正後程式碼的樣式分數
- 寫入狀態：`FIXED_STYLE_SCORE`、`FIXED_STYLE_ISSUES`
- 代理程式看到：改善或回歸

2. `compile_fix_report()`

- 讀取：所有指標 (測試、樣式、前後)
- 計算：整體狀態 (成功/部分成功/失敗)
- 寫入狀態：`FIX_REPORT`、`FIX_STATUS`
- 服務專員會看到：詳細比較

3. `exit_fix_loop()` (符合條件時顯示)

- 呼叫：僅限狀態為 `SUCCESSFUL` 時
- 組數：`escalate = True`
- 效果：這次疊代後迴圈就會結束

如果代理程式未呼叫 `exit_fix_loop`，迴圈會繼續下一次疊代。

步驟 4：瞭解 LoopAgent 結束條件

`LoopAgent` 有三種結束方式：

1. 成功結案 (透過升級)

```
# Inside any tool in the loop:
tool_context.actions.escalate = True

# Effect: Loop completes current iteration, then exits
# Use when: Fix is successful and no more attempts needed
```

範例流程：

```
Iteration 1:
  CodeFixer → generates fixes
  FixTestRunner → tests show 90% pass rate
  FixValidator → compiles report, sees PARTIAL status
  → Does NOT set escalate
  → Loop continues

Iteration 2:
  CodeFixer → refines fixes based on failures
  FixTestRunner → tests show 100% pass rate
  FixValidator → compiles report, sees SUCCESSFUL status
  → Calls exit_fix_loop() which sets escalate = True
  → Loop exits after this iteration
```

2. Max Iterations Exit

```
LoopAgent(
    name="FixAttemptLoop",
    sub_agents=[...],
    max_iterations=3 # Safety limit
)

# Effect: After 3 complete iterations, loop exits regardless of escalate
# Use when: Prevent infinite loops if fixes never succeed
```

範例流程：

```
Iteration 1: PARTIAL (continue)
Iteration 2: PARTIAL (continue)
Iteration 3: PARTIAL (but max reached)
→ Loop exits, synthesizer presents best attempt
```

3. 錯誤結束

```
# If any agent throws unhandled exception:
raise Exception("Unexpected error")

# Effect: Loop exits immediately with error state
# Use when: Critical failure that can't be recovered
```

各疊代項目的狀態演變：

每次疊代都會更新前一次嘗試的狀態：

```

# Before Iteration 1:
state = {
    "code_to_review": "def add(a,b):return a+b", # Original
    "style_score": 40,
    "test_execution_summary": {...}
}

# After Iteration 1:
state = {
    "code_to_review": "def add(a,b):return a+b", # Unchanged
    "code_fixes": "def add(a, b):\n    return a + b", # NEW
    "style_score": 40, # Unchanged
    "fixed_style_score": 100, # NEW
    "test_execution_summary": {...}, # Unchanged
    "fix_test_execution_summary": {...} # NEW
}

# Iteration 2 starts with all this state
# If fixes still not perfect, code_fixes gets overwritten

```

建議原因

escalate

改用回傳值：

```

# Bad: Using return value to signal exit
def validator_agent():
    report = compile_report()
    if report['status'] == 'SUCCESSFUL':
        return {"exit": True} # How does loop know?

# Good: Using escalate
def validator_tool(tool_context):
    report = compile_report()
    if report['status'] == 'SUCCESSFUL':
        tool_context.actions.escalate = True # Loop knows immediately
    return {"report": report}

```

優點：

- 可從任何工具運作，不限於最後一個工具
- 不會干擾回傳資料
- 清楚的語意
- 架構會處理退出邏輯

對迴圈疊代執行偵錯

如要查看每次疊代作業的進度，請按照下列步驟操作：

```
# Add to validator's state writes:
iteration_count = tool_context.state.get('loop_iteration', 0) + 1
tool_context.state['loop_iteration'] = iteration_count
tool_context.state[f'iteration_{iteration_count}_status'] = fix_status

# After loop completes, inspect:
print(f"Total iterations: {state.get('loop_iteration')}")
print(f"Iter 1: {state.get('iteration_1_status')}")
print(f"Iter 2: {state.get('iteration_2_status')}")
```

這有助於瞭解迴圈的結束時間和原因。

步驟 5：連結修正管道

👉 開啟

`code_review_assistant/agent.py`

👉 新增修正管道匯入作業 (現有匯入作業後)：

```
from google.adk.agents import LoopAgent # Add this to the existing Agent, SequentialAgent
line
from code_review_assistant.sub_agents.fix_pipeline.code_fixer import code_fixer_agent
from code_review_assistant.sub_agents.fix_pipeline.fix_test_runner import
fix_test_runner_agent
from code_review_assistant.sub_agents.fix_pipeline.fix_validator import fix_validator_agent
from code_review_assistant.sub_agents.fix_pipeline.fix_synthesizer import
fix_synthesizer_agent
```

匯入內容現在應如下所示：

```

from google.adk.agents import Agent, SequentialAgent, LoopAgent
from .config import config
# Review pipeline imports (from Module 5)
from code_review_assistant.sub_agents.review_pipeline.code_analyzer import
code_analyzer_agent
from code_review_assistant.sub_agents.review_pipeline.style_checker import
style_checker_agent
from code_review_assistant.sub_agents.review_pipeline.test_runner import test_runner_agent
from code_review_assistant.sub_agents.review_pipeline.feedback_synthesizer import
feedback_synthesizer_agent
# Fix pipeline imports (NEW)
from code_review_assistant.sub_agents.fix_pipeline.code_fixer import code_fixer_agent
from code_review_assistant.sub_agents.fix_pipeline.fix_test_runner import
fix_test_runner_agent
from code_review_assistant.sub_agents.fix_pipeline.fix_validator import fix_validator_agent
from code_review_assistant.sub_agents.fix_pipeline.fix_synthesizer import
fix_synthesizer_agent

```

👉 尋找：

```
# MODULE_6_STEP_5_CREATE_FIX_LOOP
```

👉 將該單行程式碼替換為：

```

# Create the fix attempt loop (retries up to 3 times)
fix_attempt_loop = LoopAgent(
    name="FixAttemptLoop",
    sub_agents=[
        code_fixer_agent,          # Step 1: Generate fixes
        fix_test_runner_agent,     # Step 2: Validate with tests
        fix_validator_agent        # Step 3: Check success & possibly exit
    ],
    max_iterations=3 # Try up to 3 times
)

# Wrap loop with synthesizer for final report
code_fix_pipeline = SequentialAgent(
    name="CodeFixPipeline",
    description="Automated code fixing pipeline with iterative validation",
    sub_agents=[
        fix_attempt_loop,          # Try to fix (1-3 times)
        fix_synthesizer_agent      # Present final results (always runs once)
    ]
)

```

為什麼要使用「序列」包裝「Wrap Loop」？

結構如下：

```
SequentialAgent([
    LoopAgent([fix, test, validate]), # Runs 1-3 times
    synthesizer_agent                 # Runs once
])
```

為什麼不直接：

```
LoopAgent([fix, test, validate, synthesizer]) # Bad!
```

因為無論發生多少次迴圈疊代，合成器都應在最後執行一次。如果是在迴圈內：

- 第 1 次疊代後：合成「部分」結果
- 第 2 次疊代後：再次合成 (多餘)
- 第 3 次疊代後：綜合最終

透過包裝，合成器會在所有疊代完成後查看最終狀態，並建立一份綜合報告。

👉 移除現有

root_agent

定義：

```
root_agent = Agent(...)
```

👉 尋找：

```
# MODULE_6_STEP_5_UPDATE_ROOT_AGENT
```

👉 將該單行程式碼替換為：

```
# Update root agent to include both pipelines
root_agent = Agent(
    name="CodeReviewAssistant",
    model=config.worker_model,
    description="An intelligent code review assistant that analyzes Python code and provides educational feedback",
    instruction="""You are a specialized Python code review assistant focused on helping developers improve their code quality.
```

When a user provides Python code for review:

1. Immediately delegate to CodeReviewPipeline and pass the code EXACTLY as it was provided by the user.
2. The pipeline will handle all analysis and feedback
3. Return ONLY the final feedback from the pipeline - do not add any commentary

After completing a review, if significant issues were identified:

- If style score < 100 OR tests are failing OR critical issues exist:
 - * Add at the end: "\n\n💡 I can fix these issues for you. Would you like me to do that?"
- If the user responds yes or requests fixes:
 - * Delegate to CodeFixPipeline
 - * Return the fix pipeline's complete output AS-IS

When a user asks what you can do or general questions:

- Explain your capabilities for code review and fixing
- Do NOT trigger the pipeline for non-code messages

The pipelines handle everything for code review and fixing - just pass through their final output."""

```
    sub_agents=[code_review_pipeline, code_fix_pipeline],
    output_key="assistant_response"
)
```


雙管道架構

```
Root Agent
├── CodeReviewPipeline (Module 5)
│   ├── CodeAnalyzer
│   ├── StyleChecker
│   ├── TestRunner
│   └── FeedbackSynthesizer
└── CodeFixPipeline (Module 6)
    ├── FixAttemptLoop (LoopAgent, 1-3x)
    │   ├── CodeFixer
    │   ├── FixTestRunner
    │   └── FixValidator (may set escalate)
    └── FixSynthesizer (runs once after loop)
```

為什麼要使用不同的管道？

- 「審查」是唯讀模式，「修正」則會修改程式碼 (不同考量)
- 使用者可能只想審查，不需要修正
- 修正方式取決於審查結果 (依序相依)
- 清楚劃分界線，方便測試

步驟 6：新增 Fix Synthesizer Agent

迴圈完成後，合成器會以簡單易懂的方式呈現修正結果。

👉 開啟

```
code_review_assistant/sub_agents/fix_pipeline/fix_synthesizer.py
```

👉 尋找：

```
# MODULE_6_STEP_6_FIX_SYNTHESIZER_INSTRUCTION_PROVIDER
```

👉 將該單行程式碼替換為：

```

async def fix_synthesizer_instruction_provider(context: ReadonlyContext) -> str:
    """Dynamic instruction provider that injects state variables."""
    template = """You are responsible for presenting the fix results to the user.

```

```

Based on the validation report: {final_fix_report}
Fixed code from state: {code_fixes}
Fix status: {fix_status}

```

Create a comprehensive yet friendly response that includes:

```
## 🛠️ Fix Summary
```

```
[Overall status and key improvements - be specific about what was achieved]
```

```
## 📊 Metrics
```

```

- Test Results: [original pass rate]% → [new pass rate]%
- Style Score: [original]/100 → [new]/100
- Issues Fixed: X of Y

```

```
## ✅ What Was Fixed
```

```
[List each fixed issue with brief explanation of the correction made]
```

```
## 📄 Complete Fixed Code
```

```
[Include the complete, corrected code from state - this is critical]
```

```
## 💡 Explanation of Key Changes
```

```
[Brief explanation of the most important changes made and why]
```

```
[If any issues remain]
```

```
## ⚠️ Remaining Issues
```

```
[List what still needs manual attention]
```

```
## 🔄 Next Steps
```

```
[Guidance on what to do next - either use the fixed code or address remaining issues]
```

Save the fix report using `save_fix_report` tool before presenting.

Call it with no parameters - it will retrieve the report from state automatically.

Be encouraging about improvements while being honest about any remaining issues.

Focus on the educational aspect - help the user understand what was wrong and how it was fixed.

```
"""
```

```
    return await instructions_utils.inject_session_state(template, context)
```

👉 尋找：

```
# MODULE_6_STEP_6_FIX_SYNTHESIZER_AGENT
```

👉 將該單行程式碼替換為：

```
fix_synthesizer_agent = Agent(  
    name="FixSynthesizer",  
    model=config.critic_model,  
    description="Creates comprehensive user-friendly fix report",  
    instruction=fix_synthesizer_instruction_provider,  
    tools=[FunctionTool(func=save_fix_report)],  
    output_key="fix_summary"  
)
```

👉 新增

`save_fix_report`

工具，

`tools.py`

:

👉 尋找：

```
# MODULE_6_STEP_6_SAVE_FIX_REPORT
```

👉 替換為：

```

async def save_fix_report(tool_context: ToolContext) -> Dict[str, Any]:
    """
    Saves the fix report as an artifact.

    Args:
        tool_context: ADK tool context

    Returns:
        Save status
    """
    logger.info("Tool: Saving fix report...")

    try:
        # Get the report from state
        fix_report = tool_context.state.get(StateKeys.FIX_REPORT, {})

        if not fix_report:
            return {
                "status": "error",
                "message": "No fix report found in state"
            }

        # Convert to JSON
        report_json = json.dumps(fix_report, indent=2)
        report_part = types.Part.from_text(text=report_json)

        # Generate filename
        timestamp = datetime.now().isoformat().replace(':', '-')
        filename = f"fix_report_{timestamp}.json"

        # Try to save as artifact
        if hasattr(tool_context, 'save_artifact'):
            try:
                version = await tool_context.save_artifact(filename, report_part)
                await tool_context.save_artifact("latest_fix_report.json", report_part)

                logger.info(f"Tool: Fix report saved as {filename}")

                return {
                    "status": "success",
                    "filename": filename,
                    "version": str(version),
                    "size": len(report_json)
                }
            except Exception as e:
                logger.warning(f"Could not save as artifact: {e}")

        # Fallback: store in state
        tool_context.state[StateKeys.LAST_FIX_REPORT] = fix_report

    return {
        "status": "success",

```

```

        "message": "Fix report saved to state",
        "size": len(report_json)
    }

except Exception as e:
    logger.error(f"Tool: Failed to save fix report: {e}", exc_info=True)
    return {
        "status": "error",
        "message": str(e)
    }

```

為什麼合成器會在迴圈後執行？

合成器位於迴圈「外部」：

```

SequentialAgent([
    LoopAgent([...]), # Runs 1-3 times
    synthesizer       # Runs ONCE after loop exits
])

```

也就是說：

- 在所有修正嘗試後，該元件會看到「FINAL」狀態
- 它知道發生了多少次疊代（來自狀態）
- 這份報告會提供完整資訊，而非每次疊代的報告

指令範本會參照在疊代中累積的狀態鍵：

- `{code_fixes}` - 上次嘗試的代碼
- `{final_fix_report}` - 上次驗證工具執行作業的報告
- `{fix_status}` - SUCCESSFUL/PARTIAL/FAILED

如果合成器位於迴圈內，就會使用不完整的資料多次執行。

步驟 7：測試完整修正管道

現在來看看整個迴圈的實際運作方式。

👉 啟動系統：

```
adk web code_review_assistant
```

執行 `adk web` 指令後，終端機應會顯示 ADK 網頁伺服器已啟動的輸出內容，如下所示：

```
+-----+
| ADK Web Server started                                |
|                                                       |
| For local testing, access at http://localhost:8000.  |
+-----+
```

INFO: Application startup complete.

INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)

👉 測試提示：

Please analyze the following:

```
def dfs_search_v1(graph, start, target):
    """Find if target is reachable from start."""
    visited = set()
    stack = start

    while stack:
        current = stack.pop()

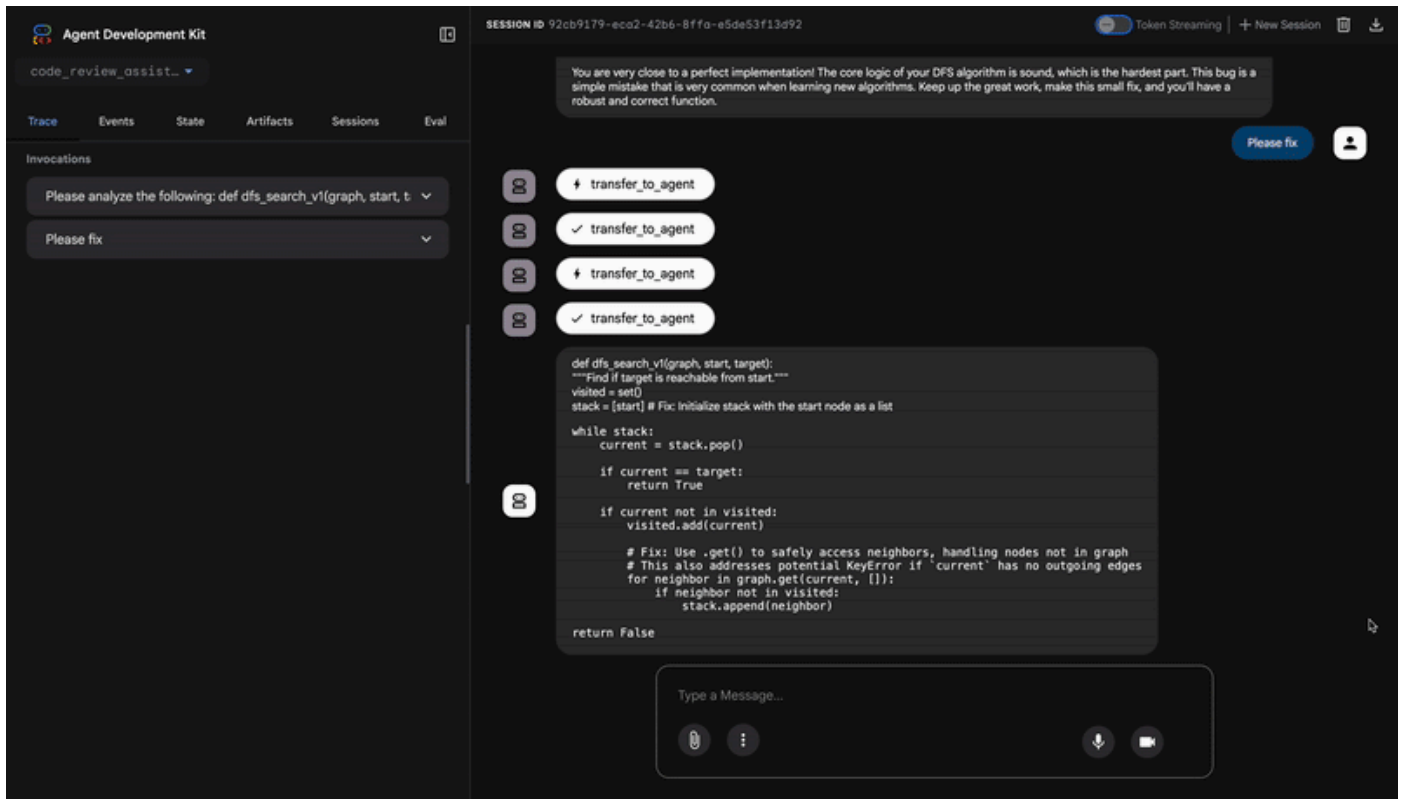
        if current == target:
            return True

        if current not in visited:
            visited.add(current)

            for neighbor in graph[current]:
                if neighbor not in visited:
                    stack.append(neighbor)

    return False
```

首先，請提交有錯誤的程式碼，觸發**審查管道**。找出瑕疵後，請要求代理「Please fix the code」(請修正程式碼)，這會觸發強大的疊代**修正管道**。



1. 初步審查 (找出安全漏洞)

這是程序的前半段。四個代理程式審查管道會分析程式碼、檢查樣式，並執行產生的測試套件。正確找出重大 `AttributeError` 和其他問題，並做出判斷：程式碼有問題，測試通過率僅 **84.21%**。

2. 自動修正 (迴圈運作)

這是最令人印象深刻的部分。要求代理程式修正程式碼時，代理程式不會只進行一項變更，這會啟動疊代式「修正和驗證迴圈」，就像勤奮的開發人員一樣：嘗試修正、徹底測試，如果不完美，就再次嘗試。

疊代 #1：首次嘗試 (部分成功)

- 修正方式：CodeFixer 代理程式會讀取初始報告，並進行最明顯的修正。這會將 `stack = start` 變更為 `stack = [start]`，並使用 `graph.get()` 來避免 `KeyError` 例外狀況。
- 驗證：TestRunner 會立即對這個新代碼重新執行完整測試套件。
- 結果：通過率大幅提升至 **88.89%**！重大錯誤已修正。不過，這些測試非常全面，因此揭露了兩個新的細微錯誤 (迴歸)，與處理 `None` 做為圖表或非清單鄰項值有關。系統會將修正標示為「部分」。

第 2 次疊代：最後潤飾 (100% 成功)

- 修正方式：由於迴圈的結束條件 (100% 通過率) 未達成，因此迴圈會再次執行。CodeFixer 現在有更多資訊，也就是兩項新的迴歸失敗。並產生最終的更健全程式碼版本，明確處理這些極端情況。
- 驗證：TestRunner 會針對最終版本的程式碼，最後一次執行測試套件。
- 結果：**100% 通過率**。所有原始錯誤和所有回歸問題都已解決。系統會將修正作業標示為「成功」，並結束迴圈。

3. 最終報告：完美分數

確認修正方式完全有效後，FixSynthesizer 代理程式會接手製作最終報告，將技術資料轉換為清楚易懂的摘要。

指標	之前	晚於	改善項目

測試通過率	84.21%	100%	▲ 15.79%
風格分數	88 / 100	98 / 100	▲ 10 分
修正錯誤	0/3	3/3	✓

✓ 最終驗證碼

以下是完整且修正後的程式碼，現在已通過所有 19 項測試，顯示修正成功：

```
def dfs_search_v1(graph, start, target):
    """Find if target is reachable from start."""
    # Handles 'None' graph input
    if graph is None:
        return False

    visited = set()
    # Fixes the critical AttributeError
    stack = [start]

    while stack:
        current = stack.pop()

        if current == target:
            return True

        if current not in visited:
            visited.add(current)

            # Safely gets neighbors to prevent KeyError
            neighbors = graph.get(current)

            if neighbors is None:
                continue

            # Validates that neighbors are iterable
            if not isinstance(neighbors, (list, set, tuple)):
                raise TypeError(
                    f"Graph value for node '{current}' is of type "
                    f"{type(neighbors).__name__}. Expected a list, set, or tuple."
                )

            for neighbor in neighbors:
                if neighbor not in visited:
                    stack.append(neighbor)

    return False
```

👉 測試完成後，請返回 Cloud Shell 編輯器終端機，然後按下 `Ctrl+C` 停止 ADK 開發人員使用者介面。

建構項目

您現在擁有完整的自動修正管道，可執行下列作業：

- ✓ 生成修正內容 - 根據審查分析結果
- ✓ 反覆驗證 - 每次嘗試修正後都會進行測試
- ✓ 自動重試 - 最多嘗試 3 次，直到成功為止
- ✓ 智慧結束 - 成功時會透過升級結束
- ✓ 追蹤改善情形 - 比較修正前後的指標
- ✓ 提供構件 - 可下載的修正報告

已掌握的關鍵概念

LoopAgent 與循序：

- 循序：依序呼叫代理
- LoopAgent：重複執行，直到符合結束條件或達到疊代次數上限
- 從 `tool_context.actions.escalate = True` 出站

各疊代項目的狀態演變：

- `CODE_FIXES` 每次疊代都會更新
- 測試結果顯示成效隨時間改善
- 驗證者會看到累計變更

多管道架構：

- 查看管道：唯讀分析 (單元 5)
- 修正迴圈：反覆修正 (模組 6 內迴圈)
- 修正管道：迴路 + 合成器 (模組 6 外側)
- 根代理程式：根據使用者意圖協調

控制流程的工具：

- `exit_fix_loop()` 組已提報
- 任何工具都可以發出迴圈完成訊號
- 將結束邏輯與代理程式指令分開

最大疊代次數安全：

- 防止無限迴圈
- 確保系統一律會回應
- 即使不完美，也會盡力呈現最佳結果

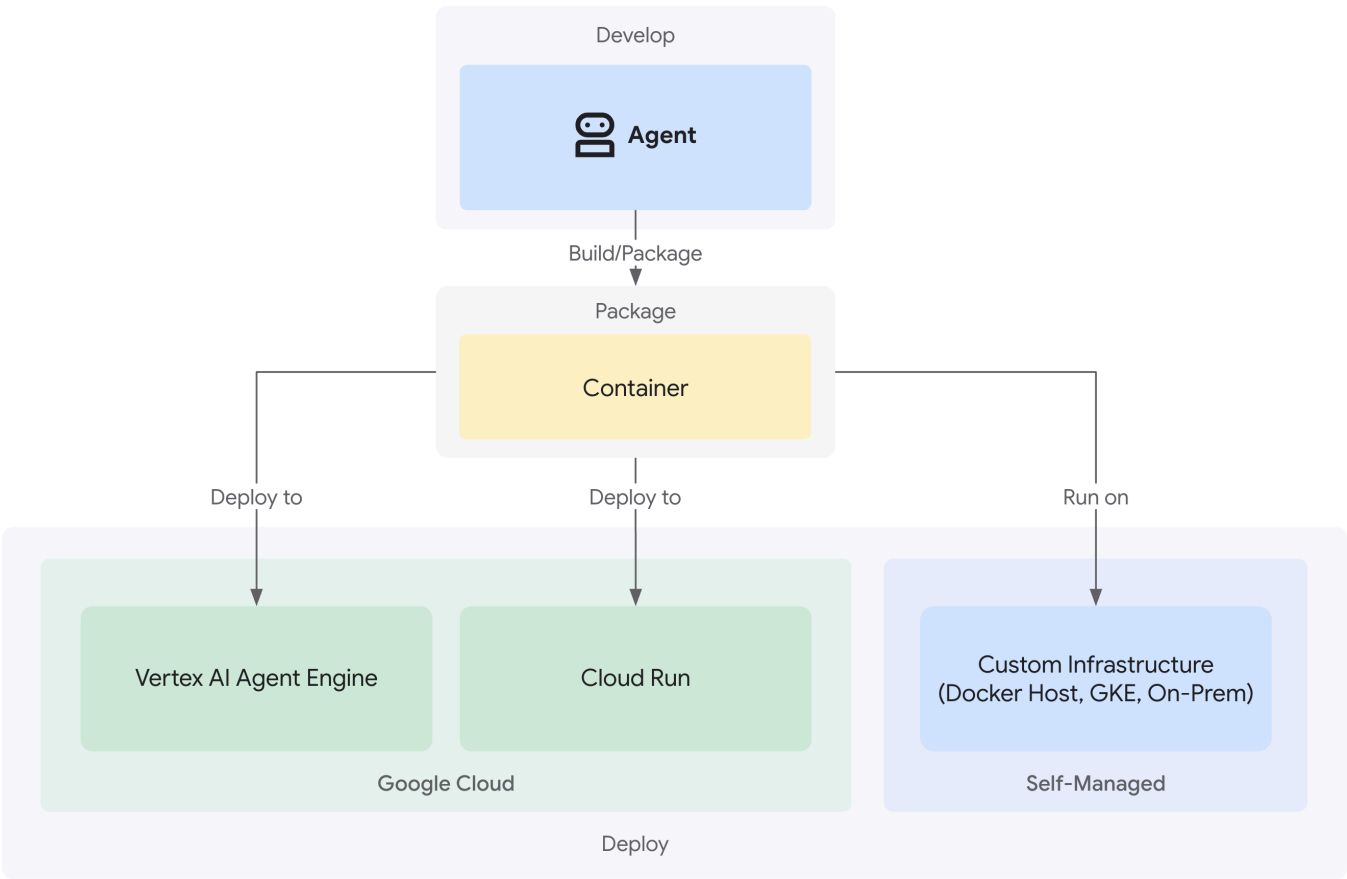
後續步驟

在最後一個單元中，您將瞭解如何將代理程式部署至正式環境：

- 使用 `VertexAiSessionService` 設定永久儲存空間
- 部署至 Google Cloud 上的 Agent Engine
- 監控及偵錯正式環境代理程式
- 擴充和可靠性的最佳做法

您已建構完整的連續和迴圈架構多代理系統。您學到的模式 (狀態管理、動態指令、工具協調和反覆改良) 都是實際代理系統中使用的正式環境技術。

7. 部署至正式環境



簡介

程式碼審查助理現在已完成，審查和修正管道可在本機運作。缺少的環節：它只會在您的電腦上執行。在本單元中，您將代理程式部署至 Google Cloud，讓團隊能夠透過持續性工作階段和正式環境等級的基礎架構存取代理程式。

白皮書內容：

- 三種部署路徑：本機、Cloud Run 和 Agent Engine
- 自動化基礎架構佈建
- 工作階段持續性策略
- 測試已部署的代理

瞭解部署選項

ADK 支援多個部署目標，各有不同的取捨：

部署路徑

因素	當地 (<code>adk web</code>)	Cloud Run (<code>adk deploy cloud_run</code>)	Agent Engine (<code>adk deploy agent_engine</code>)
複雜度	最低	中	低

工作階段持續性	僅限記憶體內 (重新啟動後會消失)	Cloud SQL (PostgreSQL)	Vertex AI 管理 (自動)
基礎架構	無 (僅限開發裝置)	容器 + 資料庫	全代管
冷啟動	不適用	100 至 2000 毫秒	100 到 500 毫秒
擴大運用	單一執行個體	自動 (歸零)	自動
費用模式	免費 (本機運算)	以要求為依據 + 免費方案	以運算為基礎
UI 支援	是 (透過 <code>adk web</code>)	是 (透過 <code>--with_ui</code>)	否 (僅限 API)
最佳用途	開發/測試	流量不穩定，需要控制成本	製作代理商

其他部署選項：Google Kubernetes Engine (GKE) 適用於需要 Kubernetes 層級控制、自訂網路或多服務協調作業的高階使用者。本程式碼研究室不會介紹 GKE 部署作業，但[ADK 部署指南](#)中已說明相關內容。

部署內容

部署至 Cloud Run 或 Agent Engine 時，系統會封裝並部署下列項目：

- 您的代理程式碼 (`agent.py` 、所有子代理程式、工具)
- 依附元件 (`requirements.txt`)
- ADK API 伺服器 (自動納入)
- 網頁介面 (僅限 Cloud Run，且指定 `--with_ui` 時)

重要差異：

- **Cloud Run**：使用 `adk deploy cloud_run` CLI (自動建構容器) 或 `gcloud run deploy` (需要自訂 Dockerfile)
- **Agent Engine**：使用 `adk deploy agent_engine` CLI (無須建構容器，直接封裝 Python 程式碼)

步驟 1：設定環境

設定 `.env` 檔案

您需要更新 `.env` 檔案 (在第 3 節中建立)，才能進行雲端部署。開啟 `.env`，然後確認/更新下列設定：

所有雲端部署作業的必要步驟：

```
# Your actual GCP Project ID (REQUIRED)
GOOGLE_CLOUD_PROJECT=your-project-id

# GCP region for deployments (REQUIRED)
GOOGLE_CLOUD_LOCATION=us-central1

# Use Vertex AI (REQUIRED)
GOOGLE_GENAI_USE_VERTEXAI=true

# Model configuration (already set)
WORKER_MODEL=gemini-2.5-flash
CRITIC_MODEL=gemini-2.5-pro
```

設定 **bucket** 名稱 (執行 **deploy.sh** 前的必要步驟)：

部署指令碼會根據這些名稱建立 bucket。立即設定：

```
# Staging bucket for Agent Engine code uploads (REQUIRED for agent-engine)
STAGING_BUCKET=gs://your-project-id-staging

# Artifact storage for reports and fixed code (REQUIRED for both cloud-run and agent-engine)
ARTIFACT_BUCKET=gs://your-project-id-artifacts
```

請將兩個 bucket 名稱中的 `your-project-id` 替換成實際專案 ID。如果這些值不存在，指令碼會建立這些值。

選用變數 (空白時會自動建立)：

```
# Agent Engine ID (populated after first deployment)
AGENT_ENGINE_ID=

# Cloud Run Database credentials (created automatically if blank)
CLOUD_SQL_INSTANCE_NAME=
DB_USER=
DB_PASSWORD=
DB_NAME=
```

指令碼會自動處理下列事項：

- 使用您指定的名稱建立測試環境和構件 bucket
- 設定服務帳戶的 IAM 權限
- 啟用必要的 Google Cloud API
- 為 Cloud Run 部署作業佈建 Cloud SQL
- 產生安全的資料庫憑證

驗證檢查

如果在部署期間發生驗證錯誤：

```
gcloud auth application-default login
gcloud config set project $GOOGLE_CLOUD_PROJECT
```

步驟 2：瞭解部署指令碼

`deploy.sh` 指令碼為所有部署模式提供統一介面：

```
./deploy.sh {local|cloud-run|agent-engine}
```

指令碼功能

基礎架構佈建：

- 啟用 API (AI Platform、Storage、Cloud Build、Cloud Trace、Cloud SQL)
- IAM 權限設定 (服務帳戶、角色)
- 建立資源 (buckets、資料庫、執行個體)
- 使用適當的旗標進行部署
- 部署後驗證

指令碼主要部分

- 設定 (第 1 至 35 行)：專案、區域、服務名稱、預設值
- 輔助函式 (第 37 至 200 行)：啟用 API、建立 bucket、設定 IAM
- 主要邏輯 (第 202 至 400 行)：模式專屬的部署自動化調度管理

步驟 3：準備 Agent Engine 適用的 Agent

部署至 Agent Engine 前，您需要 `agent_engine_app.py` 檔案，將代理程式包裝至受管理執行階段。系統已為您建立這項設定。

查看 `code_review_assistant/agent_engine_app.py`

👉 開啟檔案：

```
"""
Agent Engine application wrapper.
This file prepares the agent for deployment to Vertex AI Agent Engine.
"""

from vertexai import agent_engines
from .agent import root_agent

# Wrap the agent in an AdkApp object for Agent Engine deployment
app = agent_engines.AdkApp(
    agent=root_agent,
    enable_tracing=True,
)
```

步驟 4：部署至 Agent Engine

Agent Engine 是 **ADK** 代理的建議正式環境部署方式，因為它提供：

- 全代管基礎架構 (不必建構容器)
- 透過 `VertexAiSessionService` 內建工作階段持續性
- 從零開始自動調整資源配置
- 預設啟用 Cloud Trace 整合功能

Agent Engine 與其他部署方式的差異

幕後運作方式：

```
deploy.sh agent-engine
```

用途：

```
adk deploy agent_engine \  
  --project=$GOOGLE_CLOUD_PROJECT \  
  --region=$GOOGLE_CLOUD_LOCATION \  
  --staging_bucket=$STAGING_BUCKET \  
  --display_name="Code Review Assistant" \  
  --trace_to_cloud \  
  code_review_assistant
```

此指令會執行下列作業：

- 直接封裝 Python 程式碼 (不需 Docker 建構作業)
- 上傳至您在 `.env` 中指定的暫存值區
- 建立受管理的 Agent Engine 執行個體
- 啟用 Cloud Trace 以進行可觀測性
- 使用 `agent_engine_app.py` 設定執行階段

與會將程式碼容器化的 Cloud Run 不同，Agent Engine 會直接在受管理執行階段環境中執行 Python 程式碼，類似於無伺服器函式。

執行部署作業

從專案根目錄：

```
./deploy.sh agent-engine
```

部署階段

觀看指令碼執行下列階段：

Phase 1: API Enablement

- ✓ aiplatform.googleapis.com
- ✓ storage-api.googleapis.com
- ✓ cloudbuild.googleapis.com
- ✓ cloudtrace.googleapis.com

Phase 2: IAM Setup

- ✓ Getting project number
- ✓ Granting Storage Object Admin
- ✓ Granting AI Platform User
- ✓ Granting Cloud Trace Agent

Phase 3: Staging Bucket

- ✓ Creating gs://your-project-id-staging
- ✓ Setting permissions

Phase 4: Artifact Bucket

- ✓ Creating gs://your-project-id-artifacts
- ✓ Configuring access

Phase 5: Validation

- ✓ Checking agent.py exists
- ✓ Verifying root_agent defined
- ✓ Checking agent_engine_app.py exists
- ✓ Validating requirements.txt

Phase 6: Build & Deploy

- ✓ Packaging agent code
- ✓ Uploading to staging bucket
- ✓ Creating Agent Engine instance
- ✓ Configuring session persistence
- ✓ Setting up Cloud Trace integration
- ✓ Running health checks

這項程序需要 **5 到 10 分鐘**，因為系統會封裝代理程式，並部署至 Vertex AI 基礎架構。

儲存 Agent Engine ID

部署成功後：

```
✓ Deployment successful!
Agent Engine ID: 7917477678498709504
Resource Name: projects/123456789/locations/us-
central1/reasoningEngines/7917477678498709504
Endpoint: https://us-central1-aiplatform.googleapis.com/v1/...

⚠ IMPORTANT: Save this in your .env file:
AGENT_ENGINE_ID=7917477678498709504
```

更新

`.env`

檔案：

```
echo "AGENT_ENGINE_ID=7917477678498709504" >> .env
```

您必須提供此 ID，才能：

- 測試已部署的代理
- 稍後更新部署作業
- 存取記錄和追蹤記錄

部署內容

Agent Engine 部署作業現在包含：

- ✓ 完整審查管道 (4 位代理人)
- ✓ 完整修正管道 (迴圈 + 合成器)
- ✓ 所有工具 (AST 分析、樣式檢查、構件產生)
- ✓ 工作階段持續性 (透過 `VertexAiSessionService` 自動)
- ✓ 狀態管理 (工作階段/使用者/生命週期層級)
- ✓ 可觀測性 (已啟用 Cloud Trace)
- ✓ 自動調整資源配置基礎架構

步驟 5：測試已部署的代理程式

更新 `.env` 檔案

部署完成後，請確認 `.env` 包含：

```
AGENT_ENGINE_ID=7917477678498709504 # From deployment output
GOOGLE_CLOUD_PROJECT=your-project-id
GOOGLE_CLOUD_LOCATION=us-central1
```

執行測試指令碼

專案包含 `tests/test_agent_engine.py`，專門用於測試 Agent Engine 部署作業：

```
python tests/test_agent_engine.py
```

測試內容

1. 驗證 Google Cloud 雲端專案
2. 與已部署的代理程式建立工作階段
3. 傳送程式碼審查要求 (DFS 錯誤範例)
4. 透過伺服器推送事件 (SSE) 串流傳回回應
5. 驗證工作階段持續性和狀態管理

預期輸出

```
Authenticated with project: your-project-id
Targeting Agent Engine: projects/.../reasoningEngines/7917477678498709504

Creating new session...
Created session: 4857885913439920384

Sending query to agent and streaming response:
data: {"content": {"parts": [{"text": "I'll analyze your code..."}]}}
data: {"content": {"parts": [{"text": "**Code Structure Analysis**\n..."}]}}
data: {"content": {"parts": [{"text": "**Style Check Results**\n..."}]}}
data: {"content": {"parts": [{"text": "**Test Results**\n..."}]}}
data: {"content": {"parts": [{"text": "**Final Feedback**\n..."}]}}

Stream finished.
```

驗證項目清單

- ☒ 執行完整審查管道 (所有 4 位專員)
- ☒ 串流回應會逐步顯示輸出內容
- ☒ 工作階段狀態會在要求之間保留
- ☒ 沒有驗證或連線錯誤
- ☒ 工具呼叫成功執行 (AST 分析、樣式檢查)
- ☒ 儲存構件 (可存取評分報告)

替代方案：部署至 Cloud Run

雖然建議使用 Agent Engine 簡化正式環境部署作業，但 Cloud Run 提供更多控制權，並支援 ADK 網頁使用者介面。本節提供總覽說明。

使用 Cloud Run 的時機

如果您需要下列功能，請選擇 Cloud Run：

- 供使用者互動的 ADK 網頁 UI
- 完整控管容器環境
- 自訂資料庫設定
- 與現有 Cloud Run 服務整合

Cloud Run 部署作業的運作方式

幕後運作方式：

```
deploy.sh cloud-run
```

用途：

```
adk deploy cloud_run \
  --project=$GOOGLE_CLOUD_PROJECT \
  --region=$GOOGLE_CLOUD_LOCATION \
  --service_name="code-review-assistant" \
  --app_name="code_review_assistant" \
  --port=8080 \
  --with_ui \
  --artifact_service_uri="gs://$ARTIFACT_BUCKET" \
  --trace_to_cloud \
  code_review_assistant
```

此指令會執行下列作業：

- 使用代理程式碼建構 Docker 容器
- 推送至 Google Artifact Registry
- 以 Cloud Run 服務的形式部署
- 包括 ADK 網頁版 UI (`--with_ui`)
- 設定 Cloud SQL 連線 (初始部署後由指令碼新增)

與 Agent Engine 的主要差異：Cloud Run 會將程式碼容器化，並需要資料庫來保存工作階段，而 Agent Engine 會自動處理這兩項作業。

Cloud Run 部署指令

```
./deploy.sh cloud-run
```

不同之處

基礎架構：

- 容器化部署 (Docker 由 ADK 自動建構)
- 使用 Cloud SQL (PostgreSQL) 持久儲存工作階段
- 指令碼自動建立的資料庫，或使用現有執行個體

工作階段管理：

- 使用 `DatabaseSessionService`，而非 `VertexAiSessionService`
- 需要 `.env` 中的資料庫憑證 (或自動產生)
- 狀態會保留在 PostgreSQL 資料庫中

UI 支援：

- 可透過 `--with_ui` 旗標使用網頁 UI (由指令碼處理)
- 存取位置：`https://code-review-assistant-xyz.a.run.app`

學習成果

正式版部署作業包含：

✔ 透過 `deploy.sh` 指令碼

自動佈建 ✔ 受管理基礎架構 (Agent Engine 會處理資源調度、持續性、監控)

- ✓ 持續性狀態 (跨所有記憶體層級：工作階段/使用者/生命週期)
- ✓ 安全憑證管理 (自動產生和 IAM 設定)
- ✓ 可擴充架構 (從零到數千名並行使用者)
- ✓ 內建可觀測性 (已啟用 Cloud Trace 整合)
- ✓ 正式版錯誤處理和復原

已掌握的關鍵概念

部署準備：

- `agent_engine_app.py`：使用 `AdkApp` 包裝代理程式，以供 Agent Engine 使用
- `AdkApp` 會自動設定 `VertexAiSessionService` 以確保資料持續存在
- 已透過 `enable_tracing=True` 啟用追蹤功能

部署指令：

- `adk deploy agent_engine`：封裝 Python 程式碼，不含容器
- `adk deploy cloud_run`：自動建構 Docker 容器
- `gcloud run deploy`：使用自訂 Dockerfile 的替代方案

部署選項：

- Agent Engine：全代管，最快可投入正式環境
- Cloud Run：提供更多控制權，支援網頁版 UI
- GKE：進階 Kubernetes 控制項 (請參閱 [GKE 部署指南](#))

代管服務：

- Agent Engine 會自動處理工作階段持續性
- Cloud Run 需要設定資料庫 (或自動建立)
- 兩者都支援透過 GCS 儲存構件

工作階段管理：

- 代理引擎：`VertexAiSessionService` (自動)
- Cloud Run：`DatabaseSessionService` (Cloud SQL)
- 本機：`InMemorySessionService` (暫時性)

服務專員已上線

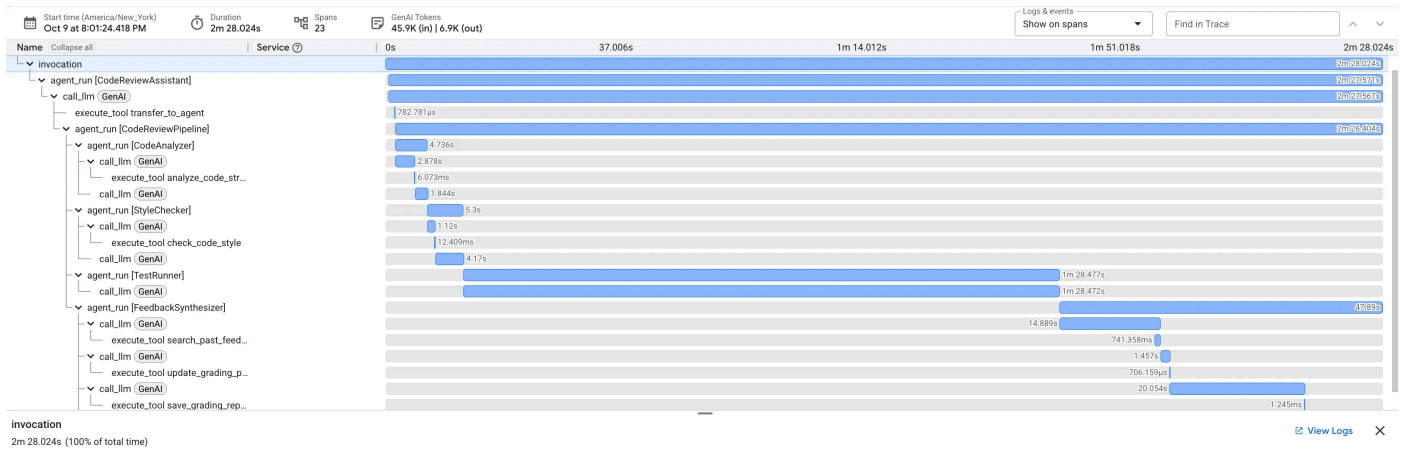
程式碼審查助理現在是：

- 透過 HTTPS API 端點存取
- **Persistent**：狀態會在重新啟動後保留
- 可擴充，自動因應團隊成長
- 可觀測，並提供完整的要求追蹤記錄
- 透過指令碼部署作業維護

後續步驟 在第 8 個單元中，您將瞭解如何使用 Cloud Trace 掌握代理程式的效能、找出審查和修正管道中的瓶頸，以及縮短執行時間。

步驟 8 · 約 0 分鐘

8. 正式版觀測



簡介

程式碼審查助理現已部署完畢，並在 Agent Engine 的正式環境中執行。但要怎麼知道成效如何？你能回答下列重要問題嗎？

- 服務專員是否及時回覆？
- 哪些作業最慢？
- 修正迴圈是否有效完成？
- 效能瓶頸在哪裡？

如果沒有觀測能力，您就只能盲目行事。您在部署期間使用的 `--trace-to-cloud` 旗標會自動啟用 Cloud Trace，讓您全面掌握代理程式處理的每項要求。這項功能可將偵錯作業從猜測轉為鑑識分析。

在本單元中，您將瞭解如何解讀追蹤記錄、掌握代理程式的效能特徵，以及根據確切證據找出最佳化領域。

瞭解追蹤記錄和範圍

什麼是追蹤記錄？

「追蹤記錄」是代理程式處理單一要求時的完整時間軸。從使用者傳送查詢到系統傳送最終回應，這段期間的所有內容都會記錄下來。每項追蹤記錄都會顯示：

- 要求的總時間長度
- 所有已執行的作業
- 作業之間的關係 (父項/子項關係)
- 每項作業的開始和結束時間

什麼是 Span？

「時距」代表追蹤記錄中的單一工作單元。程式碼審查助理的常見範圍類型：

- `agent_run`：執行代理 (根代理或子代理)
- `call_llm`：對語言模型的要求
- `execute_tool`：執行工具函式
- `state_read` / `state_write`：狀態管理作業
- `code_executor`：執行含有測試的程式碼

時距具有下列特性：

- **名稱**：代表的作業
- **時間長度**：完成作業所需的時間
- **屬性**：模型名稱、權杖計數、輸入/輸出內容等中繼資料
- **狀態**：成功或失敗
- **父項/子項關係**：哪些作業觸發了哪些作業

自動檢測

使用 `--trace-to-cloud` 部署時，ADK 會自動檢測下列項目：

- 每次代理程式叫用和子代理呼叫
- 所有 LLM 要求 (含詞元數量)
- 輸入/輸出內容的工具執行作業
- 狀態作業 (讀取/寫入)
- 在修正管道中進行迴圈疊代
- 錯誤狀況和重試

無須變更程式碼：ADK 的執行階段內建追蹤功能。

步驟 1：存取 Cloud Trace 探索器

在 **Google Cloud** 控制台中開啟 **Cloud Trace**：

1. 前往 [Cloud Trace 探索工具](#)
2. 從下拉式選單中選取專案 (應已預先選取)
3. 您應該會在第 7 堂課的模組中看到測試的追蹤記錄

如果還沒看到追蹤記錄：

您在單元 7 中執行的測試應該會產生追蹤記錄。如果清單為空白，請產生一些追蹤記錄資料：

```
python tests/test_agent_engine.py
```

等待 1 到 2 分鐘，追蹤記錄就會顯示在控制台中。

您看到的內容

Trace 探索工具會顯示：

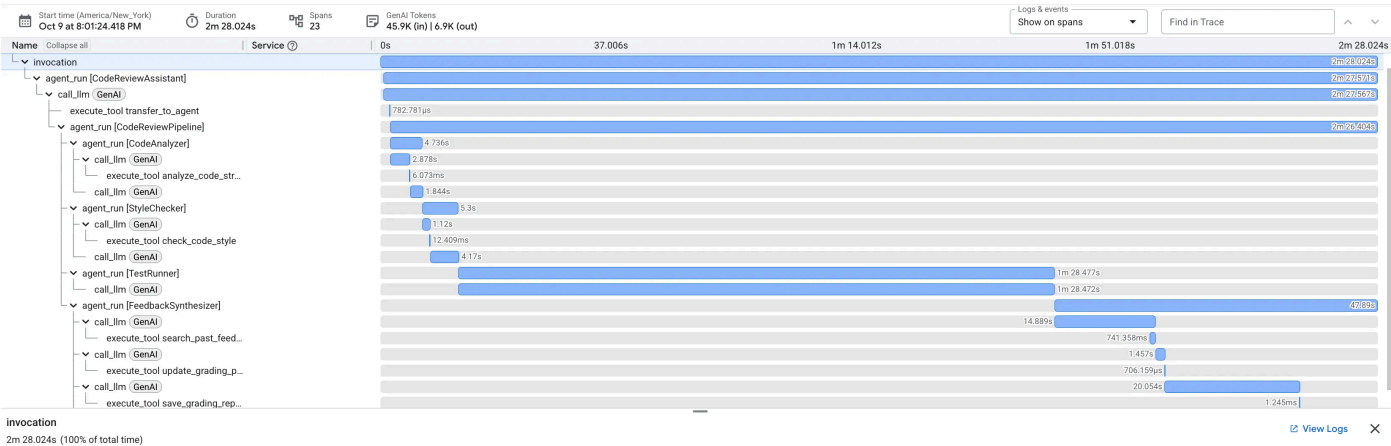
- **追蹤記錄清單**：每個資料列代表一項完整要求
- **時間軸**：發生要求的時間
- **時間長度**：每個要求所花費的時間
- **要求詳細資料**：時間戳記、延遲時間、範圍計數

這是您的正式版流量記錄，每次與代理程式互動都會建立追蹤記錄。

步驟 2：檢查審查管道追蹤記錄

按一下清單中的任何追蹤記錄，即可開啟瀑布圖檢視畫面。

畫面上會顯示甘特圖，其中包含完整的執行時間軸。根 `invocation` 時距代表整個要求。下方是每個子代理、工具和 LLM 呼叫的範圍。



解讀瀑布圖：找出瓶頸

每個長條代表一個範圍。水平位置代表開始時間，長度則代表所需時間。這會立即顯示代理程式的時間分配情形。

上述追蹤記錄的重要洞察：

- **總延遲時間**：整個要求耗時 **2 分 28 秒**。
- **子代理程式細目**：
 - **Code Analyzer**：4.7 秒
 - **Style Checker**：5.3 秒
 - **Test Runner**：**1 分 28 秒**
 - **Feedback Synthesizer**：47.9 秒
- **重要路徑分析**：**Test Runner** 代理程式是明顯的效能瓶頸，約占總要求時間的 **59%**。

這項曝光度非常強大。您不必猜測時間花費在哪裡，而是有具體證據顯示，如果需要針對延遲時間進行最佳化，**Test Runner** 就是明顯的目標。

檢查權杖用量，提高成本效益

Cloud Trace 不僅會顯示時間，還會擷取每次 LLM 呼叫的權杖用量，藉此揭露費用。

按一下

`call_llm`

追蹤記錄中的時距。詳細資料窗格會顯示 `llm.usage.prompt_tokens` 和 `llm.usage.completion_tokens` 的屬性。

GenAI Tokens

1.1K (in), 490 (out)

Related Attributes

gen_ai.request.model	gemini-2.5-flash
gen_ai.response.finish_reasons	stop
gen_ai.system	gcp.vertex.agent
gen_ai.usage.input_tokens	1123
gen_ai.usage.output_tokens	490

您可以藉此：

- **追蹤精細層級的費用：**準確掌握每個代理程式和工具消耗的權杖數量。
- **找出最佳化機會：**如果代理程式使用的詞元數量異常高，或許可以藉此機會調整提示詞，或改用較小且更具成本效益的模型來執行特定任務。

步驟 3：分析修正管道追蹤記錄

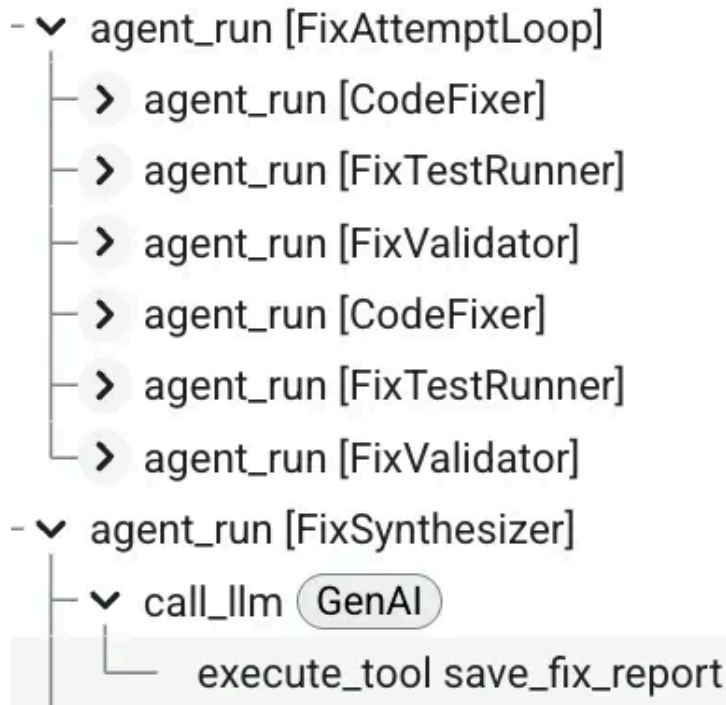
修正管道較為複雜，因為其中包含 `LoopAgent`。Cloud Trace 可讓您輕鬆瞭解這種疊代行為。

找出範圍名稱包含「`FixAttemptLoop`」的追蹤記錄。

如果沒有，請執行測試腳本，並在系統詢問是否要修正程式碼時，回答「是」。

檢查迴圈結構

追蹤記錄檢視畫面會清楚呈現迴圈的執行情況。如果修正迴圈在成功前執行了兩次，您會在 `FixAttemptLoop` 範圍下方看到兩個巢狀 `loop_iteration` 範圍，每個範圍都包含 `CodeFixer`、`FixTestRunner` 和 `FixValidator` 代理程式的完整週期。



迴圈追蹤的主要觀察結果：

- **反覆修正過程可見：**您可以在 `loop_iteration: 1` 中看到系統嘗試修正，然後驗證修正結果，但由於結果不盡完美，因此系統會在 `loop_iteration: 2` 中再次嘗試。
- **可測量收斂情形：**您可以比較每次疊代的持續時間和結果，瞭解系統如何收斂至正確的解決方案。
- **簡化偵錯程序：**如果迴圈執行疊代次數已達上限，但仍失敗，您可以檢查每次疊代範圍內的狀態和代理程式行為，診斷修正項目無法收斂的原因。

這類詳細資訊有助於瞭解及偵錯實際工作環境中複雜的具狀態迴圈行為。

步驟 4：你發現了什麼

成效模式

從檢查追蹤記錄中，您現在可以取得資料驅動的洞察資訊：

審查管道：

- **主要瓶頸：** `Test Runner` 代理 (尤其是程式碼執行和以 LLM 為基礎的測試生成) 是審查過程中耗時最長的部分。
- **快速作業：** 確定性工具 (`analyze_code_structure`) 和狀態管理作業速度極快，不會造成效能問題。

修正管道：

- **收斂率：** 您會發現大部分修正作業會在 1 到 2 次疊代中完成，確認迴圈架構有效。
- **累進成本：** 由於 LLM 脈絡會隨著先前失敗嘗試的資訊而擴大，後續疊代可能需要較長時間。

費用驅動因素：

- **詞元消耗量：** 您可以找出哪些代理程式 (例如合成器) 需要最多詞元，並判斷使用功能更強大但成本較高的模型是否合理。

如何查看問題

在正式環境中查看追蹤記錄時，請注意下列事項：

- **異常長的追蹤記錄**：表示效能回歸或出現非預期的迴圈行為。
- **失敗的範圍** (標示為紅色)：指出失敗的確切作業。
- **迴圈迭代次數過多** (超過 2 次)：可能表示修正產生邏輯有問題。
- **符記數量過高**：醒目顯示提示最佳化或模型選取變更的機會。

目前所學內容

透過 Cloud Trace，您現在瞭解如何：

- ✓ **要求流程視覺化**：查看循序和迴圈式管道的完整執行路徑。
- ✓ **找出效能瓶頸**：使用瀑布圖找出最慢的作業，並取得相關硬體資料。
- ✓ **分析迴圈行為**：觀察迭代代理程式在多次嘗試後，如何收斂至解決方案。
- ✓ **追蹤權杖費用**：檢查 LLM 跨度，以精細監控及最佳化權杖用量。

已掌握的關鍵概念

- **追蹤記錄和時距**：可觀測性的基本單位，代表要求和其中的作業。
- **瀑布式分析**：解讀甘特圖，瞭解執行時間和依附元件。
- **找出重要路徑**：找出決定整體延遲時間的作業序列。
- **精細的可觀測性**：不只能掌握時間，還能瞭解每個作業的權杖計數等中繼資料，這些資料都會由 ADK 自動記錄。

後續步驟

繼續探索 Cloud Trace：

- 定期監控追蹤記錄，及早發現問題
- 比較追蹤記錄，找出效能衰退問題
- 根據追蹤資料做出最佳化決策
- 依時間長度篩選，找出速度緩慢的要求

進階可觀測性 (選用)：

- 將追蹤記錄匯出至 BigQuery，進行複雜的分析 ([文件](#))
 - 在 Cloud Monitoring 中建立自訂資訊主頁
 - 設定效能下降快訊
 - 將追蹤記錄與應用程式記錄相關聯
-

9. 結論：從原型設計到正式環境 建構項目

您只用了七行程式碼，就建構出可供正式環境使用的 AI 代理系統：

```
# Where we started (7 lines)
agent = Agent(
    model="gemini-2.5-flash",
    instruction="Review Python code for issues"
)

# Where we ended (production system)
- Two distinct multi-agent pipelines (review and fix) built from 8 specialized agents.
- An iterative fix loop architecture for automated validation and retries.
- Real AST-based code analysis tools for deterministic, accurate feedback.
- Robust state management using the "constants pattern" for type-safe communication.
- Fully automated deployment to a managed, scalable cloud infrastructure.
- Complete, built-in observability with Cloud Trace for production monitoring.
```

已掌握的主要架構模式

模式	導入作業	對正式環境的影響
工具整合	AST 分析、樣式檢查	實際驗證，不只是 LLM 的意見
循序管道	檢查 → 修正工作流程	可預測且可偵錯的執行作業
迴圈架構	使用結束條件進行反覆修正	自我改進，直到成功為止
狀態管理	常數模式、三層記憶體	型別安全且可維護的狀態處理作業
正式版部署作業	透過 deploy.sh 部署至 Agent Engine	代管式可擴充基礎架構
觀測能力	整合 Cloud Trace	全面掌握生產行為

從追蹤記錄取得生產環境洞察資訊

- Cloud Trace 資料揭露了重要洞察：
- ✔ 找出瓶頸：TestRunner 的 LLM 呼叫作業佔據了大部分的延遲時間
 - ✔ 工具效能：AST 分析執行時間為 100 毫秒 (極佳)
 - ✔ 成功率：修正迴圈會在 2 到 3 次疊代內收斂
 - ✔ 權杖用量：每次審查約 600 個權杖，每次修正約 1800 個權杖

這些深入分析資料可做為持續改善的依據。

清除資源 (選用)

完成實驗後，如要避免產生費用，請按照下列步驟操作：

刪除 Agent Engine 部署作業：

```
import vertexai

client = vertexai.Client( # For service interactions via client.agent_engines
    project="PROJECT_ID",
    location="LOCATION",
)

RESOURCE_NAME = "projects/{PROJECT_ID}/locations/{LOCATION}/reasoningEngines/{RESOURCE_ID}"

client.agent_engines.delete(
    name=RESOURCE_NAME,
    force=True, # Optional, if the agent has resources (e.g. sessions, memory)
)
```

刪除 Cloud Run 服務 (如已建立)：

```
gcloud run services delete code-review-assistant \
    --region=$GOOGLE_CLOUD_LOCATION \
    --quiet
```

刪除 Cloud SQL 執行個體 (如已建立)：

```
gcloud sql instances delete your-project-db \
    --quiet
```

清理儲存空間值區：

```
gsutil -m rm -r gs://your-project-staging
gsutil -m rm -r gs://your-project-artifacts
```

後續步驟

完成基礎設定後，請考慮進行下列強化作業：

1. 新增其他語言：擴充工具，支援 JavaScript、Go、Java
2. 與 GitHub 整合：自動審查 PR
3. 實作快取：縮短常見模式的延遲時間
4. 新增專用代理程式：安全掃描、效能分析
5. 啟用 A/B 測試：比較不同模型和提示
6. 匯出指標：將追蹤記錄傳送至專門的可觀測性平台

重點回顧

1. 從簡單開始，快速疊代：七行程式碼即可在可管理的步驟中投入生產
2. 工具優於提示：實際的 AST 分析優於「請檢查是否有錯誤」
3. 狀態管理很重要：常數模式可避免錯別字錯誤
4. 迴圈需要結束條件：請務必設定最大疊代次數和升級
5. 透過自動化部署：deploy.sh 會處理所有複雜作業
6. 觀測能力是不可或缺的：您無法改善無法評估的事物

持續學習的資源

- [ADK 說明文件](#)
- [進階 ADK 模式](#)
- [Agent Engine 指南](#)
- [Cloud Run 說明文件](#)
- [Cloud Trace 說明文件](#)

繼續你的歷程

您不只建構了程式碼審查助理，還掌握了建構任何正式環境 AI 代理的模式：

- ✅ 包含多個專用代理的複雜工作流程
- ✅ 整合實際工具，提供真正實用的功能
- ✅ 部署至正式環境，並妥善監控
- ✅ 進行狀態管理，確保系統可維護

這些模式的規模從簡單的助理到複雜的自主系統都有。您在這裡建立的基礎，將有助於您處理日益複雜的代理程式架構。

歡迎瞭解如何開發正式版 AI 代理程式。程式碼審查助理只是起點。
